



COMSATS University, Islamabad Pakistan

MediFusion Vision

By

AIZA KHADIM	CIIT/FA22-BCS-010/ISB
AREEBA NIAZI	CIIT/FA22-BCS-014/ISB
AREEHA NAYAB	CIIT/FA22-BCS-015/ISB

Supervisor

Dr. Rasool Bukhsh

***Bachelor of Science in Computer Science / Software
Engineering (2022-2026)***

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University, Islamabad Pakistan

MediFusion Vision

**A project presented to
COMSATS University, Islamabad**

**In partial fulfillment
of the requirement for the degree of**

***Bachelors of Science in Computer Science / Software
Engineering (2022-2026)***

By

AIZA KHADIM	CIIT/FA22-BCS-010/ISB
AREEBA NIAZI	CIIT/FA22-BCS-014/ISB
AREEHA NAYAB	CIIT/FA22-BCS-015/ISB

Table Of Contents

1 Chapter 3: Design and Architecture	1
3.1 System Architecture Overview	1
3.1.1 Purpose	2
3.2 Conceptual Architecture Diagram	Error! Bookmark not defined.
3.2.1 Technologies and Services	3
3.3 Architecture Style / Pattern	4
3.4 Design Models	6
3.4.1 Design Models for Object Oriented Development Approach	6
Activity Diagrams:	6
1. AI Diagnostic Process:	6
2. Appointment Booking Process:	8
3. Virtual Consultation Process:	9
4. Doctor Registration Process:	9
5. Self-Learning Feedback Process:	10
6. Scan Management Process:	12
7. Payment Processing Process:	13
8. Admin System Monitoring Process:	14
Class Diagram:	14
Sequence Diagrams:	Error! Bookmark not defined.
1. AI Diagnostic Execution:	16
2. Appointment Booking with Payment:	18
3. Virtual Consultation Initiation:	19
4. Patient Scan Upload & Validation:	20
5. Doctor Registration Approval:	21
6. Self-Sufficient AI Error Flagging:	22
State Transition Diagrams:	24
1. Appointment Object Lifecycle:	24
2. Doctor Registration Object Lifecycle:	25
3. Diagnosis Report Object Lifecycle:	25
4. Payment Transaction Object Lifecycle:	26
3.5 Data Design	27
2 Chapter 4: Implementation	29
4.1 Project Methodology & Algorithms	30
4.1.1 Project Methodology (Step-by-Step Approach)	30
4.1.2 Algorithm	32
4.2 Training Results & Model Evaluation (Mandatory for AI/ML/Data Science Projects)	34

4.3 Security Techniques (if applicable)	41
4.4 External APIs/SDKs	42
4.5 User Interface	43
1.1.1 Login Screen 4.5.2 Home Screen	43
5.3.3 Assignee Dashboard	44
5.3.4 New Complaint	44
4.6 Deployment	45
2 Chapter 5: Testing and Evaluation	46
5.1 Unit Testing	46
5.2 Functional Testing	49
5.3 Business Rules Testing	53
5.4 Integration Testing	59

List of Figures:

Figure 1: Conceptual Architecture Diagram.....	3
Figure 2: Activity Diagram for AI Diagnostic Process	7
Figure 3: Activity Diagram for Appointment Booking Process	8
Figure 4: Activity Diagram for Virtual Consultation Process	9
Figure 5: Activity Diagram for Doctor Registration Process	10
Figure 6: Activity Diagram for Self Learning Feedback Process.....	11
Figure 7: Activity Diagram for Scan Management Process	12
Figure 8: Activity Diagram for Payment Processing Process.....	13
Figure 9: Activity Diagram for Admin System Monitoring Process.....	14
Figure 10: Class Diagram	15
Figure 11: Sequence Diagram for AI Diagnostic Execution	17
Figure 12: Sequence Diagram for Appointment Booking with Payment	18
Figure 13: Sequence Diagram for Virtual Consultation Initiation.....	19
Figure 14: Sequence Diagram for Patient Scan Upload & Validation	20
Figure 15: Sequence Diagram for Doctor Registration Approval	21
Figure 16: Sequence Diagram for Feedback Loop	23
Figure 17: State Diagram for Appointment Object Lifecycle	24
Figure 18: State Diagram for Doctor Registration.....	25
Figure 19: State Diagram for Diagnosis Report Object Lifecycle.....	26
Figure 20: State Diagram for Payment Transaction Object Lifecycle.....	27

1 Chapter 3: Design and Architecture

This chapter presents the high-level architectural design of the **MediFusion Vision** system. It describes how the system’s AI-driven diagnostic components, multimodal processing pipelines, and user-facing modules are structured and interact to fulfill the functional and non-functional requirements defined in the SRS. The chapter details the system architecture, selected architectural design patterns, and the UML-based design models used to represent internal and external interactions. Additionally, data-flow structures, storage design, and a comprehensive data dictionary are included to clarify how medical images, reports, user records, and analytical outputs move through the system. Altogether, these design artifacts form a bridge between requirement specifications and the implementation phase, ensuring reliability, scalability, and a clear blueprint for developing the MediFusion Vision platform.

3.1 System Architecture Overview

The **MediFusion Vision** system adopts a modern, cloud-enabled, AI-centric architectural design that integrates multimodal medical image processing, secure data management, and intelligent decision-support services. The system is structured into four major layers: **Frontend Layer (Client Applications)**, **Backend/API Layer**, **AI Engine Layer**, and **Database/Storage Layer**, each serving a specialized role while communicating through secure RESTful and stream-based interfaces.

The **Frontend Layer** consists of a responsive web application (React.js) and optionally a cross-platform mobile interface (Flutter), designed to facilitate interactions for patients, doctors, radiologists, and administrators. The client communicates with the backend via HTTPS, supporting image uploads, appointment scheduling, AI result viewing, report generation, and tele-consultations. All UI flows follow medical-grade usability standards with accessibility support for diverse users.

The **Backend/API Layer**, built on **Node.js** and **Express.js**, manages authentication, authorization, session management, record handling, appointment management, payment workflows, and communication services. Following a structured **Controller–Service–Model architecture**, the backend encapsulates business logic while exposing well-defined APIs to the frontend. It also coordinates requests with the AI Engine Layer and integrates with third-party services such as payment gateways, email/SMS notifications, and the video-consultation module. The **AI Engine Layer** is implemented using **Python, PyTorch, and specialized medical-imaging libraries**. This layer performs preprocessing, classification, segmentation, multimodal fusion, and explainability visualization (Grad-CAM, SHAP). It operates as a microservice, communicating with the backend through REST or gRPC channels. The layer is optimized for GPU-accelerated inference to ensure real-time or near-real-time diagnostic outputs. It also supports a feedback pipeline for human-in-the-loop model retraining.

The **Database and Storage Layer** uses a hybrid model: **PostgreSQL** for structured data (users, appointments, medical history, reports, payments) and **cloud object storage** for DICOM/X-ray/CT/MRI files and generated explainability heatmaps. This separation ensures fast querying,

scalability, and high-integrity medical data storage. Database access is further regulated through ORM-based models, strict schema validation, and audit logging.

Security spans across all layers through **JWT-based authentication, role-based access control (RBAC), AES-256 encrypted data storage, TLS/HTTPS communication, audit logs, and HIPAA-aligned privacy safeguards**. Additional protection is provided via input sanitization, secure API gateways, MFA options, and encrypted medical-image transfer.

Overall, the architecture offers a robust, scalable, and medically compliant foundation for deploying MediFusion Vision as an AI-powered diagnostic and virtual-care ecosystem, ensuring high performance, modularity, and trustworthiness.

3.1.1 Purpose

The purpose of this section is to provide a high-level conceptual understanding of the **MediFusion Vision** system architecture, showing how its major components and technologies function together to deliver an integrated, secure, and AI-powered healthcare platform. It highlights the placement of each core module such as the client applications, backend services, AI inference layer, database systems, authentication services, admin console, and third-party integrations and explains how data flows across them. This architectural overview helps stakeholders visualize the system at a macro level, ensuring clarity, maintainability, scalability, and alignment with industry standards for medical software and intelligent diagnostic systems.

3.2 Conceptual Architecture Diagram

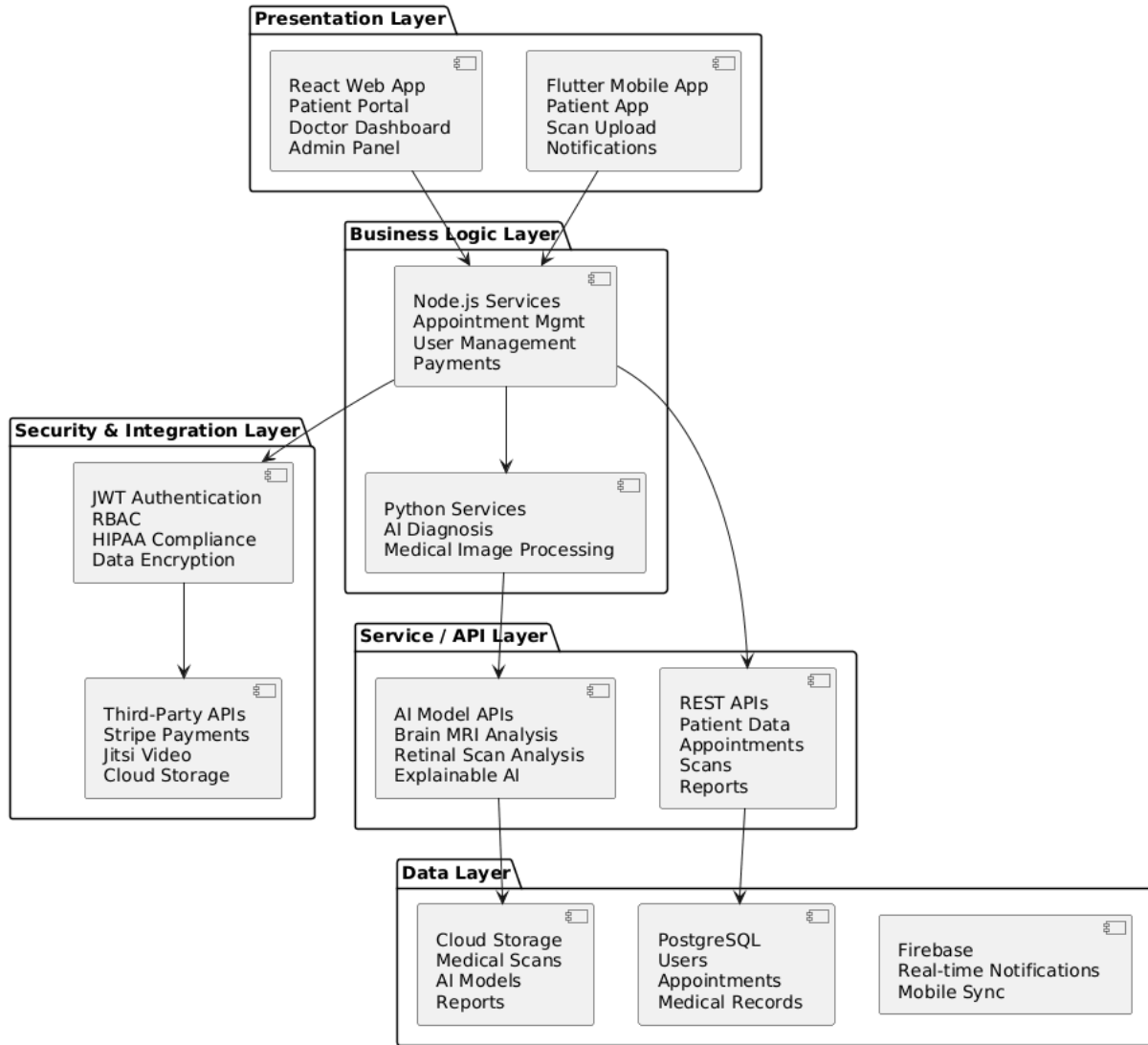


Figure 1: Conceptual Architecture Diagram

3.2.1 Technologies and Services

Table 3.1 Technologies, Components, and Security Mechanisms Used in the System Architecture

Component	Description	Technology Used	Security Mechanism
Frontend App	Patient & doctor interfaces for scan upload, reports, appointments, and dashboard.	Flutter / React.js	SSL/TLS, Input Validation
Backend API	Handles system logic, AI calls, appointments, report generation, notifications, and routing.	Node.js (Express) / Django REST	JWT Authentication, Rate Limiting

Database Layer	Stores users, reports, appointments, logs, and metadata.	PostgreSQL	RBAC, Encrypted Connections
Admin Panel	Dashboard for managing users, feedback, retraining, reports, and monitoring system activity.	React.js	2FA, Admin Authorization
Authentication Service	Verifies identity and roles of patients, doctors, and admins using tokens.	JWT	RBAC, Token Expiry Policies
AI Diagnostic Engine	Processes medical scans and produces diagnosis through trained deep learning models.	PyTorch	Model Access Control, Execution Sandbox
Explainability Engine (XAI)	Generates Grad-CAM/SHAP heatmaps for transparency.	Python (Grad-CAM, SHAP)	Secure Temporary Storage
Cloud Storage	Stores MRI scans, heatmaps, and reports.	AWS S3 / GCP Storage	AES-256 Encryption, Signed URLs
Notification Service	Sends email/SMS alerts for readiness of reports or appointments.	Twilio, Nodemailer	API Key Protection
Payment Gateway	Handles service payments and teleconsultation fees.	Stripe	Tokenization, PCI-DSS compliance

3.3 Architecture Style / Pattern

The **MediFusion Vision** system adopts a **Layered Architecture Pattern** combined with an **MVC-aligned modular structure** to support scalability, maintainability, and secure AI-powered medical diagnostics. This architectural style ensures a clean separation of concerns between the user interface, application services, AI diagnostic engine, and data storage layers. The system's design is optimized for healthcare workflows involving Patients, Doctors, and Admins, while integrating advanced diagnostic models, explainable AI outputs, and medical record management. At the macro level, MediFusion Vision uses a **three-tier architecture** composed of:

1. Presentation Layer (Frontend – Web & Mobile Interfaces)

This layer provides role-based interfaces for Patients, Doctors, and Admins. It manages all user interactions such as:

- User authentication & multi-role login
- Patient dashboards, scan uploads, and medical history views
- Doctor dashboards with diagnostic results, notes, and consultations
- Admin controls for system management, user roles, and monitoring
- Multi-language (English/Urdu) support
- Accessibility-focused UI components (readable fonts, simple navigation)

The frontend interacts with backend services through **secure RESTful APIs** over HTTPS. All medical images (MRI/retinal) uploaded by users are processed through the backend pipeline before reaching the AI Diagnostic Engine.

2. Application Layer (Backend / API Services + AI Diagnostic Engine)

This layer contains all **business logic, AI operations, validation, workflows, and process orchestration**. It aligns with MVC structure:

Models: Represent core entities defined in the SRS such as:

- User (Patient/Doctor/Admin)
- Medical Records
- MRI/Retinal Images
- Diagnostic Reports
- Appointments
- Payments
- Feedback & Logs

Controllers: Implement system behaviors defined in the SRS, including:

- User registration, login, role verification
- Scan submission and preprocessing
- AI diagnostic requests (Brain tumor, Alzheimer's, DR, Glaucoma, AMD)
- Explainable AI generation (heatmaps, region highlights)
- Report creation and history tracking
- Appointment scheduling
- Payment processing workflow
- Admin approval & management tasks

Routes / API Endpoints: Map external requests to controller logic for:

- Authentication & role-based access
- Image upload & diagnostic processing
- Report retrieval
- Virtual consultation APIs
- Admin system functions
- Payment gateway communication
- Patient/Doctor dashboard data queries

AI Diagnostic Engine: A separate internal module that handles:

- MRI/Retinal image preprocessing
- AI model inference (disease classification / severity detection)
- Result confidence scoring
- Explainable AI overlays (Grad-CAM style heatmaps)
- Model latency requirement

Support Services:

- Logging & audit trail
- Notification services (email/SMS-based updates)
- Security utilities: input validation, encryption, token verification

This layer ensures **high cohesion and loose coupling**, allowing AI models to be upgraded independently without affecting UI or database logic.

3. Data Layer (Database & Storage)

This layer stores all persistent data outlined in the SRS:

- User profiles, roles, permissions
- Medical history & previous diagnostic reports
- MRI/retinal image metadata

- Appointments and consultation records
- Admin logs & system audit history
- Feedback and usage analytics

The database ensures:

- Data Encryption (AES-256)
- Access control aligned with healthcare data confidentiality
- High availability and reliability for medical workflows

Large file storage (MRI scans, retinal images, reports) is handled through optimized secure storage.

3.4 Design Models

3.4.1 Design Models for Object Oriented Development Approach

The MediFusion Vision system is built using object-oriented principles, with design models that visualize complex workflows and system interactions between patients, doctors, and administrators.

Activity Diagrams:

Activity diagrams were created for the core, non-trivial workflows that involve multiple actors and complex business processes. The following activity diagrams were developed:

1. AI Diagnostic Process:

Patient uploads medical scan, system validates file and routes to appropriate AI model, doctor reviews AI results and explainability visuals, then verifies or corrects diagnosis to generate final report.

Activity diagram for AI Diagnostic Process is given below:

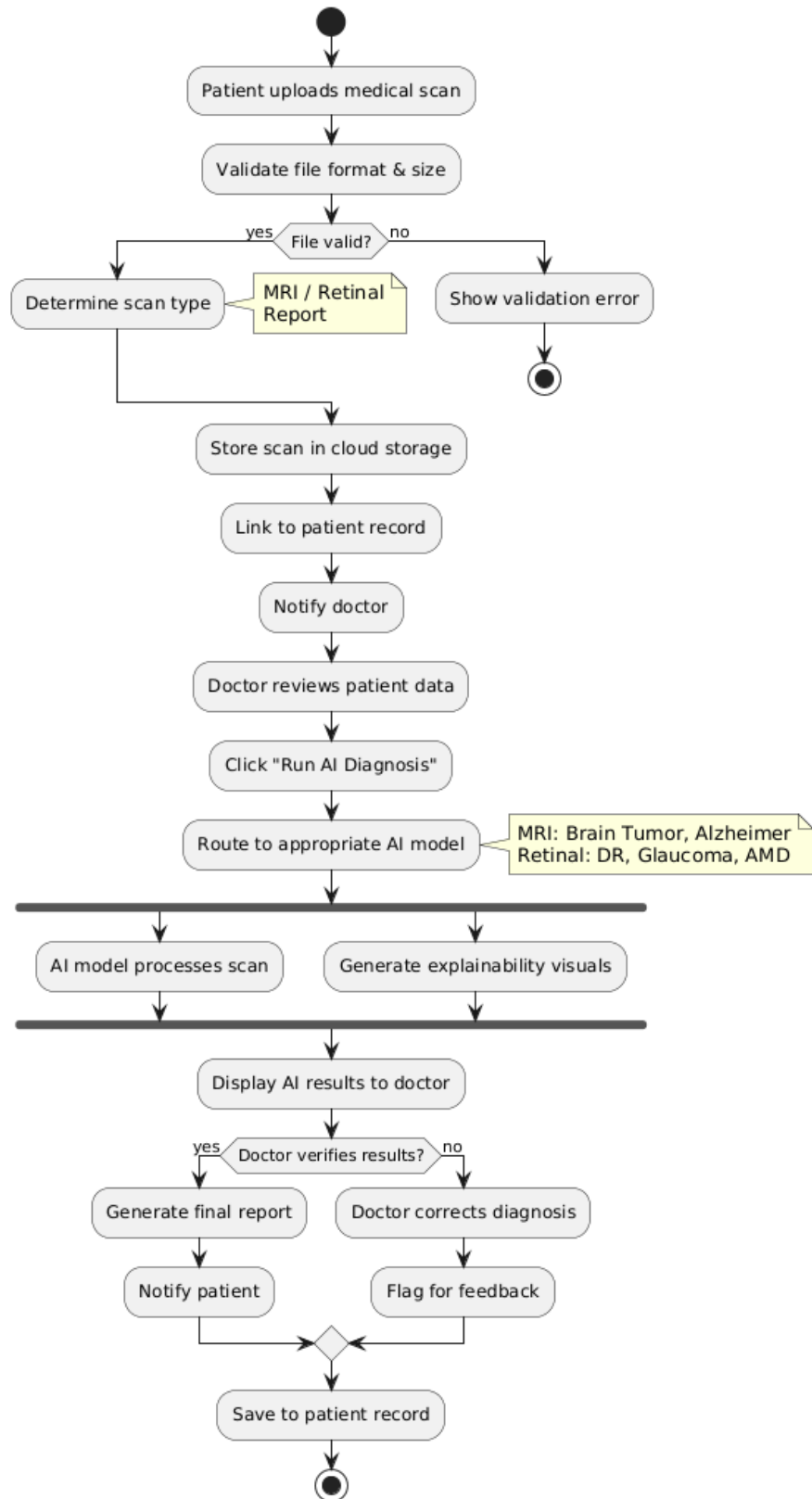


Figure 2: Activity Diagram for AI Diagnostic Process

2. Appointment Booking Process:

Patient searches for doctors by specialization, selects available time slot, provides consultation reason, system processes payment and confirms booking with automatic notifications to both parties.

Activity diagram for Appointment Booking Process is given below:

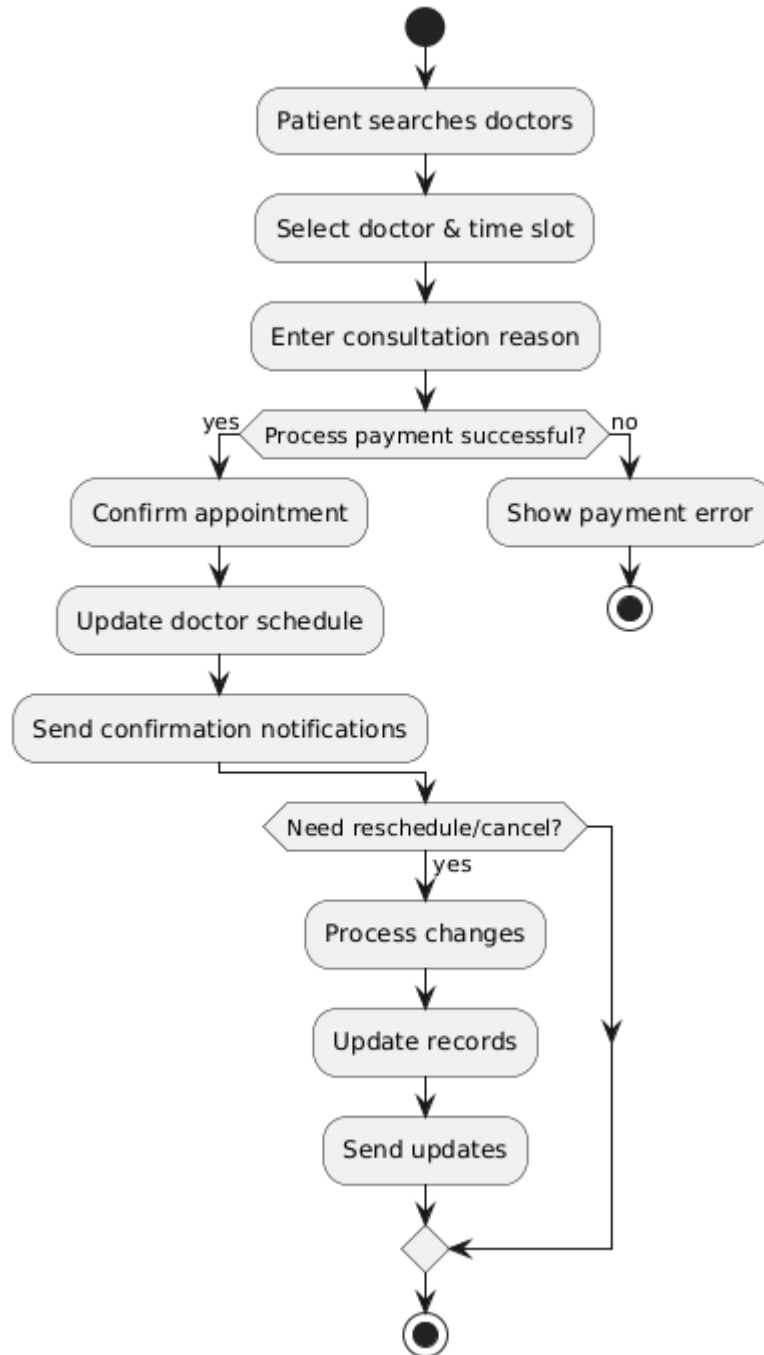


Figure 3: Activity Diagram for Appointment Booking Process

3. Virtual Consultation Process:

System generates secure video link for scheduled appointment, doctor and patient join consultation, AI Note-Taker transcribes conversation in real-time, doctor reviews and finalizes consultation notes.

Activity diagram for Virtual Consultation Process is given below:

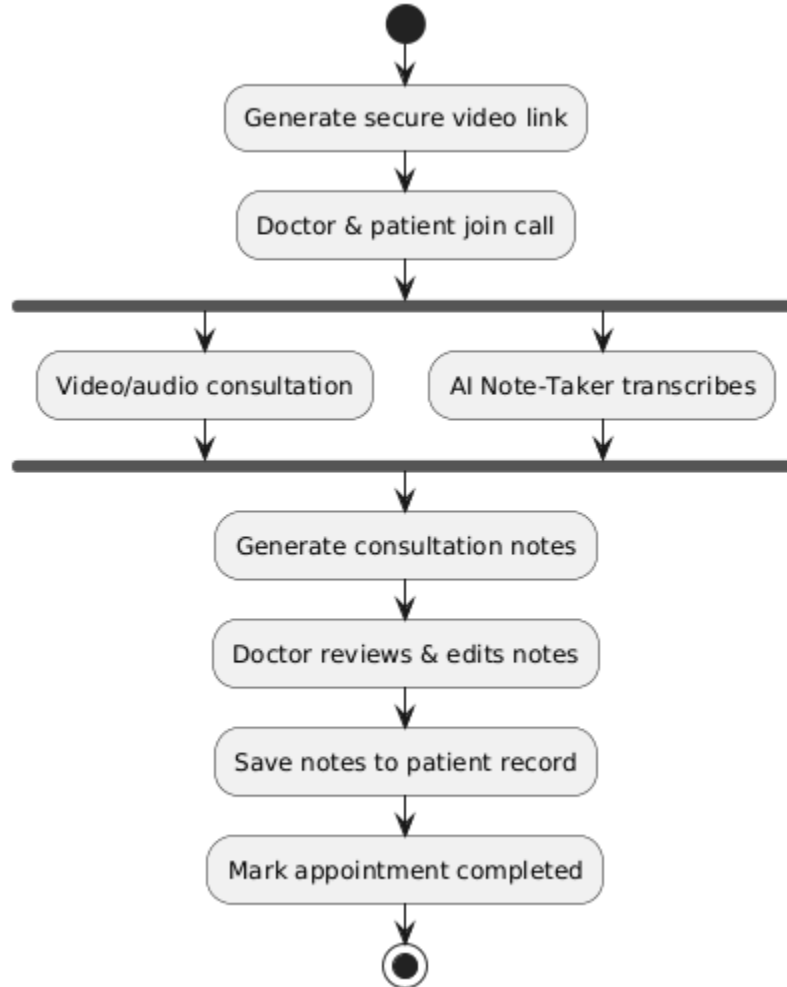


Figure 4: Activity Diagram for Virtual Consultation Process

4. Doctor Registration Process:

Doctor submits registration with qualifications and experience, admin reviews credentials and approves or rejects application, system activates approved accounts and notifies doctor. Activity diagram for Doctor Registration Process is given below:

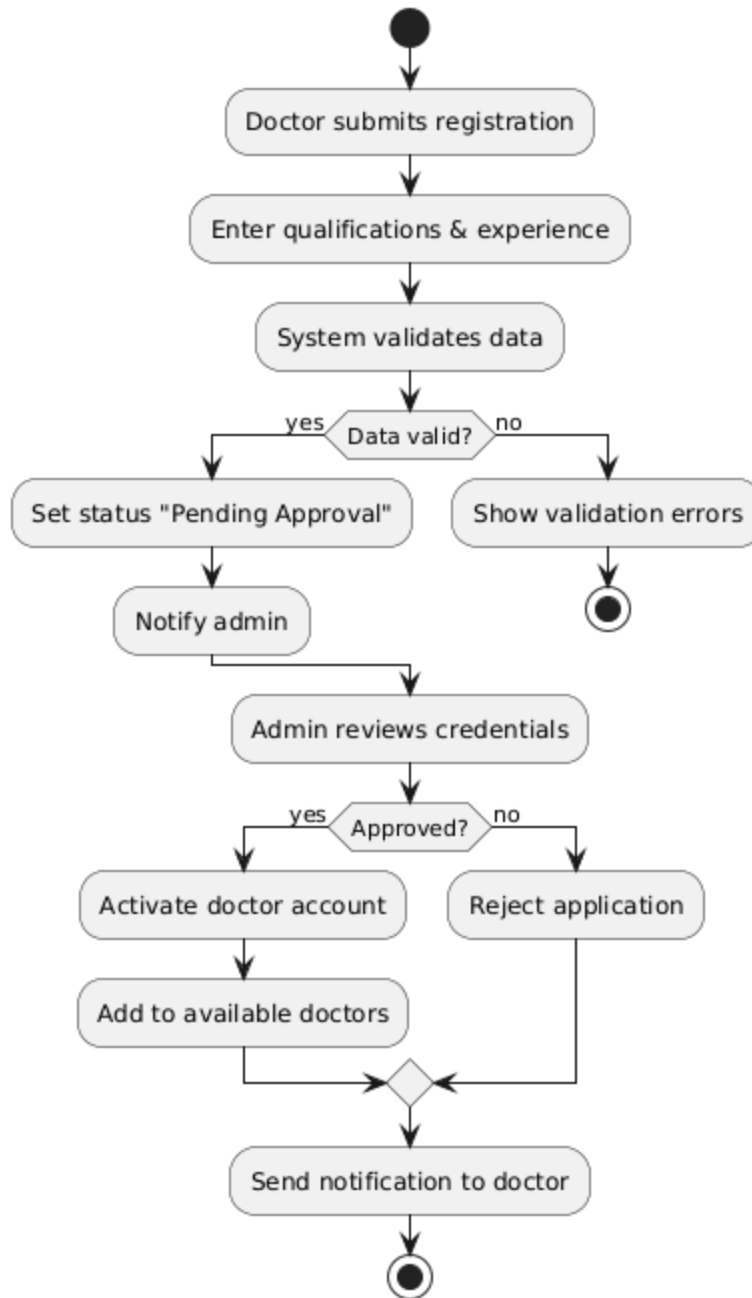


Figure 5: Activity Diagram for Doctor Registration Process

5. Self-Learning Feedback Process:

Doctor flags incorrect AI diagnosis with corrections, system checks case quality and analyzes consensus patterns, automatically validates cases and adds to training dataset for model improvement.

Activity diagram for Self-Learning Feedback Process is given below:

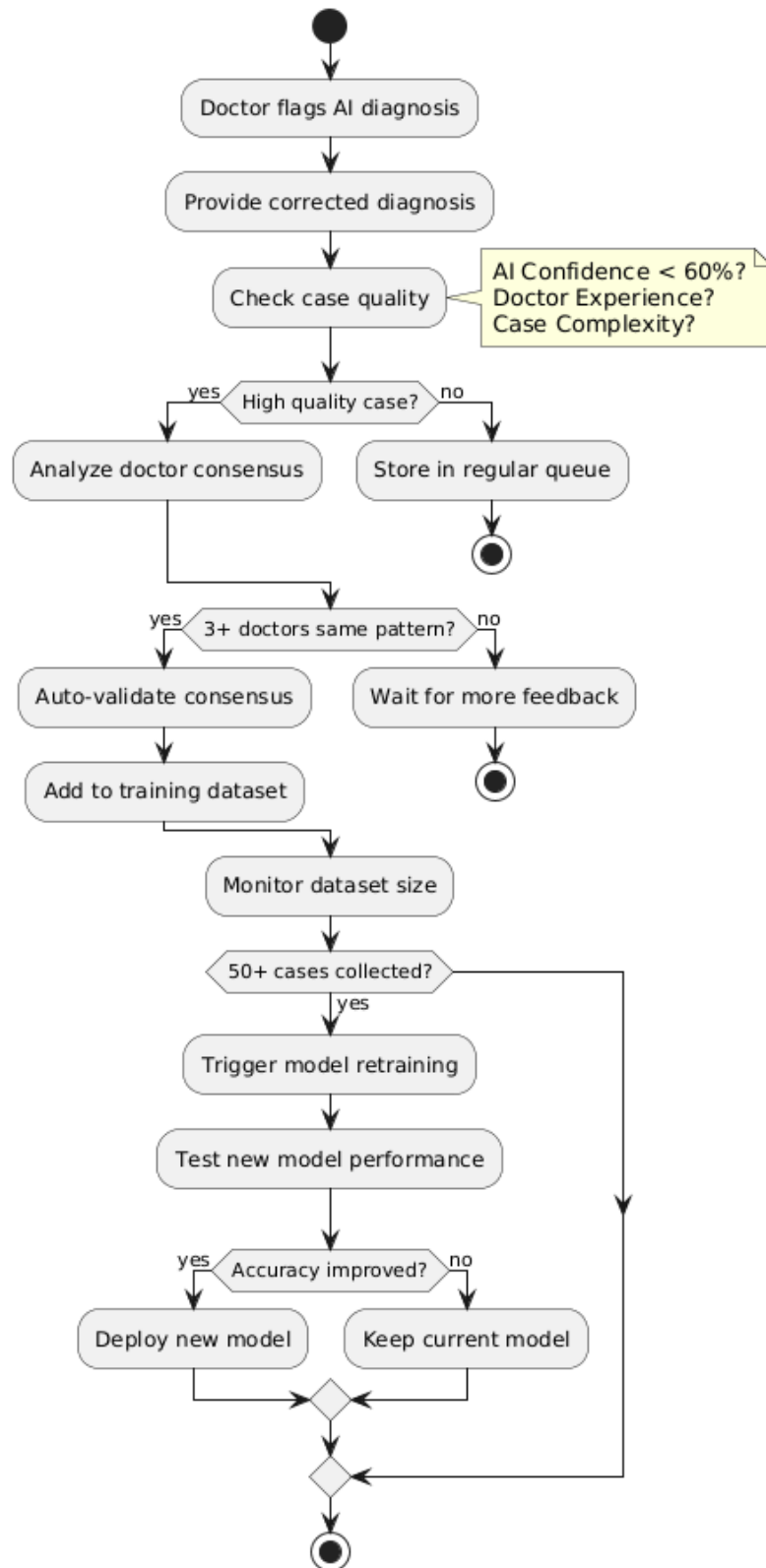


Figure 6: Activity Diagram for Self Learning Feedback Process

6. Scan Management Process:

Patient or admin selects scan type and uploads file, system validates format and size, encrypts and stores in cloud storage, links to patient record and updates repository.

Activity diagram for Scan Management Process is given below:

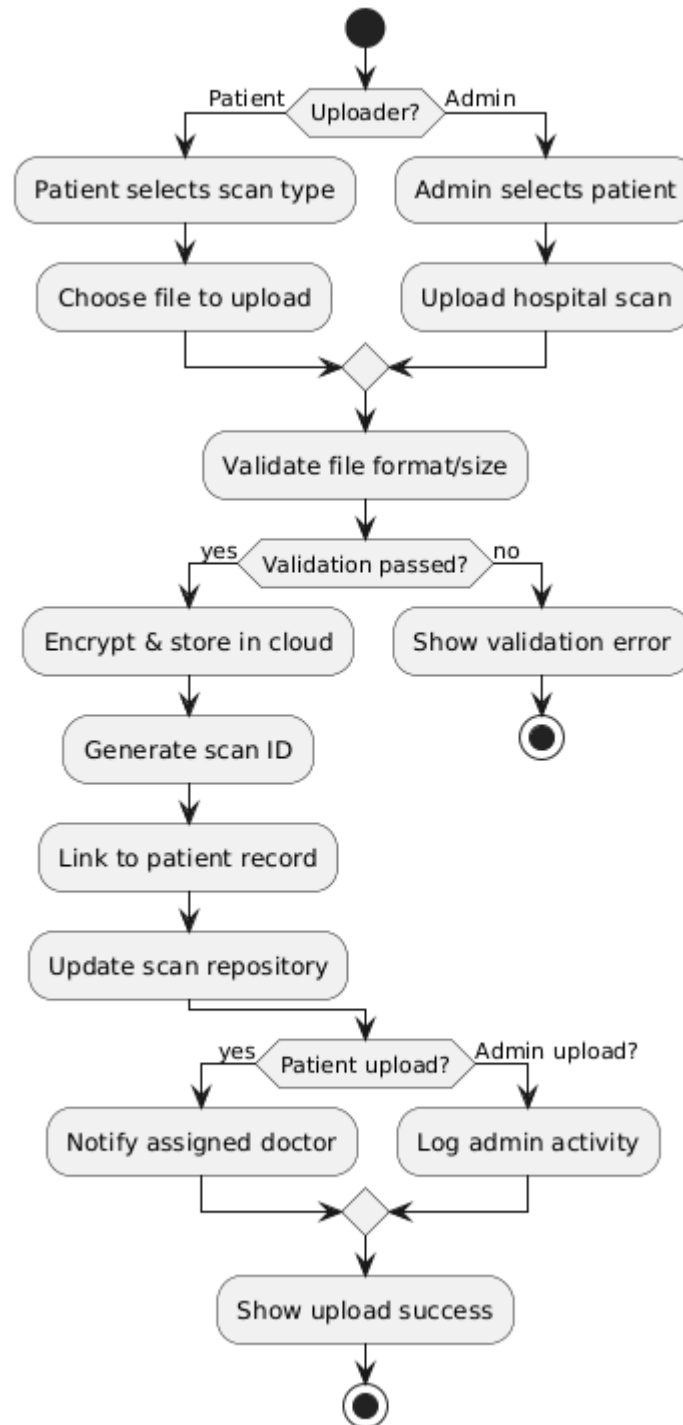


Figure 7: Activity Diagram for Scan Management Process

7. Payment Processing Process:

Patient selects appointment slot, system calculates consultation fee and redirects to payment gateway, processes card or wallet payment, confirms transaction and generates receipt upon success.

Activity diagram for Payment Processing Process is given below:

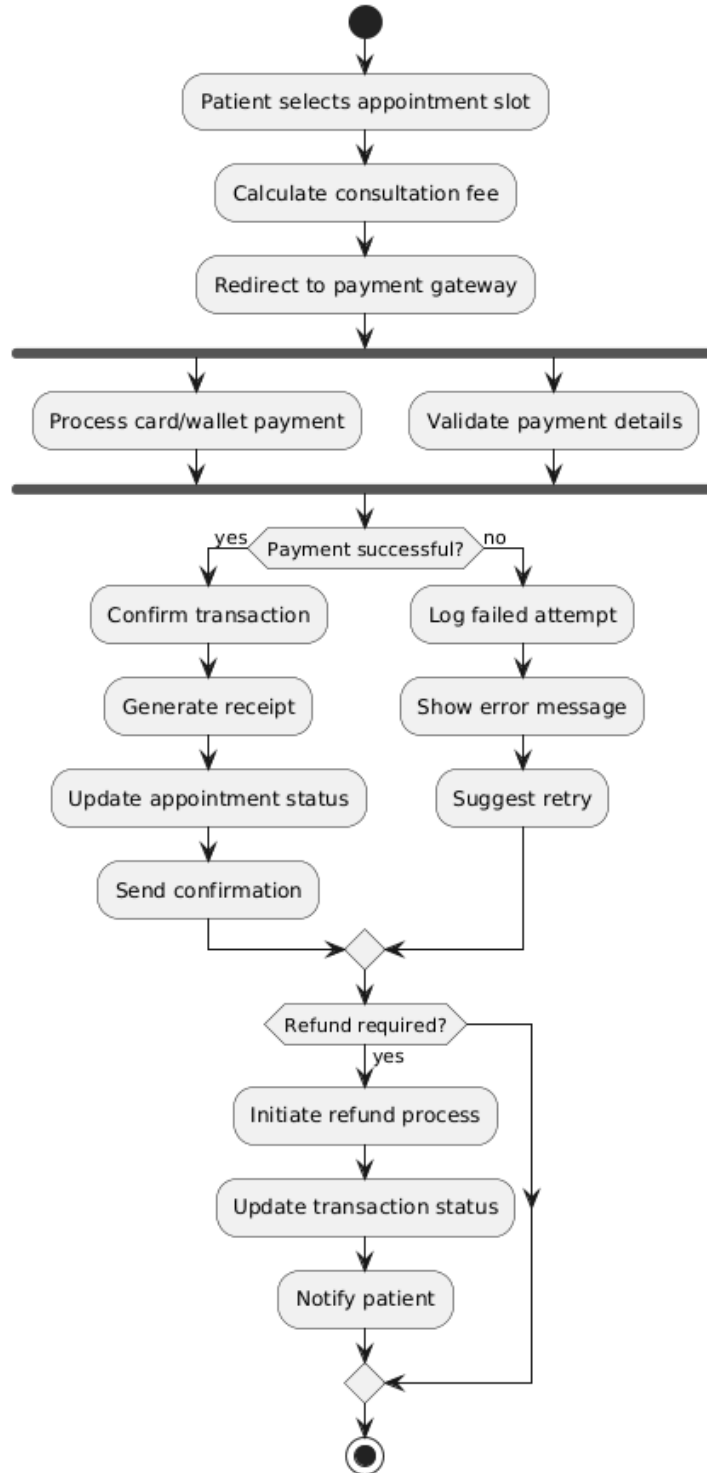


Figure 8: Activity Diagram for Payment Processing Process

8. Admin System Monitoring Process:

Admin logs into dashboard, views system overview and performance metrics, monitors AI model accuracy and user feedback patterns, generates reports and updates system settings. Activity diagram for Admin System Monitoring Process is given below:

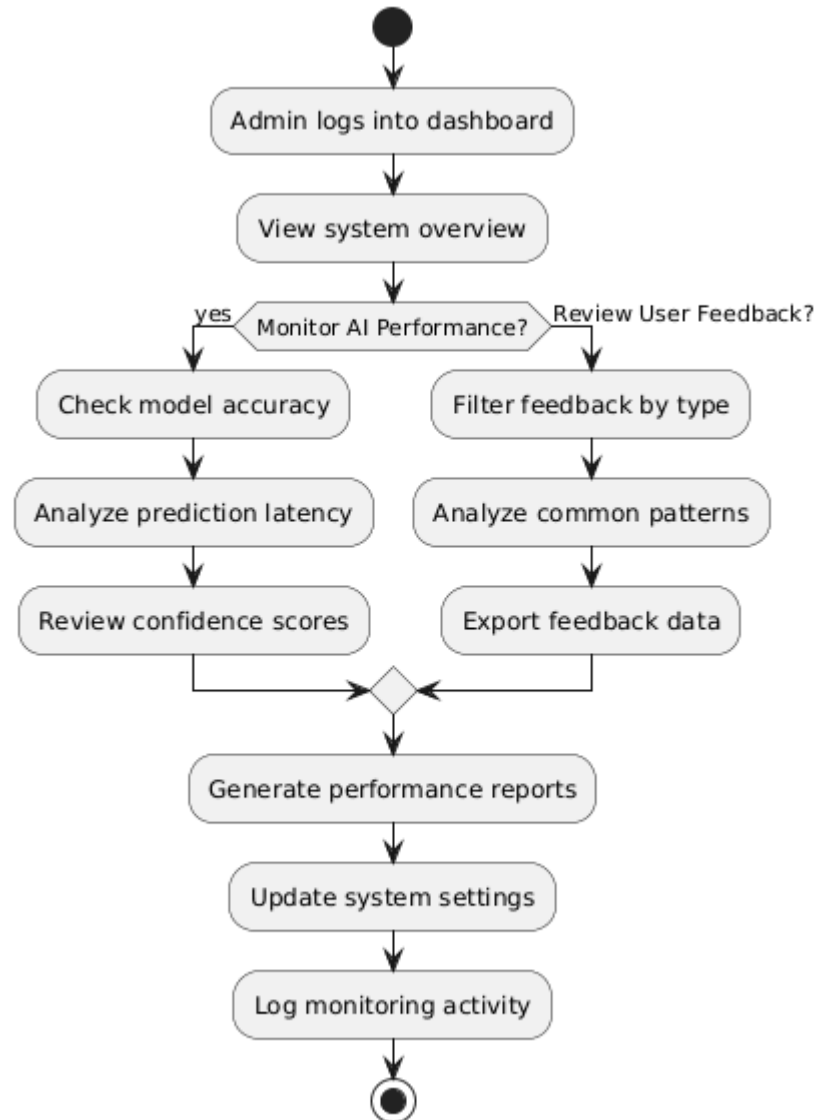


Figure 9: Activity Diagram for Admin System Monitoring Process

Class Diagram:

The class diagram represents MediFusion Vision's complete object-oriented structure with a User inheritance hierarchy where Patient, Doctor, and Admin classes share common attributes while having specialized properties. Core entities include Appointment for healthcare scheduling with virtual and physical options, MedicalScan supporting both patient and admin uploads, DiagnosisReport combining AI analysis with doctor verification, and PaymentTransaction for secure financial processing. The self-learning feedback system with QualityChecker, ConsensusEngine, and TrainingManager enables automated AI improvement through doctor

corrections and consensus validation. Relationships illustrate complete workflows where patients book appointments with specialized doctors, medical scans generate AI-powered diagnostic reports, and feedback mechanisms drive continuous system intelligence enhancement.

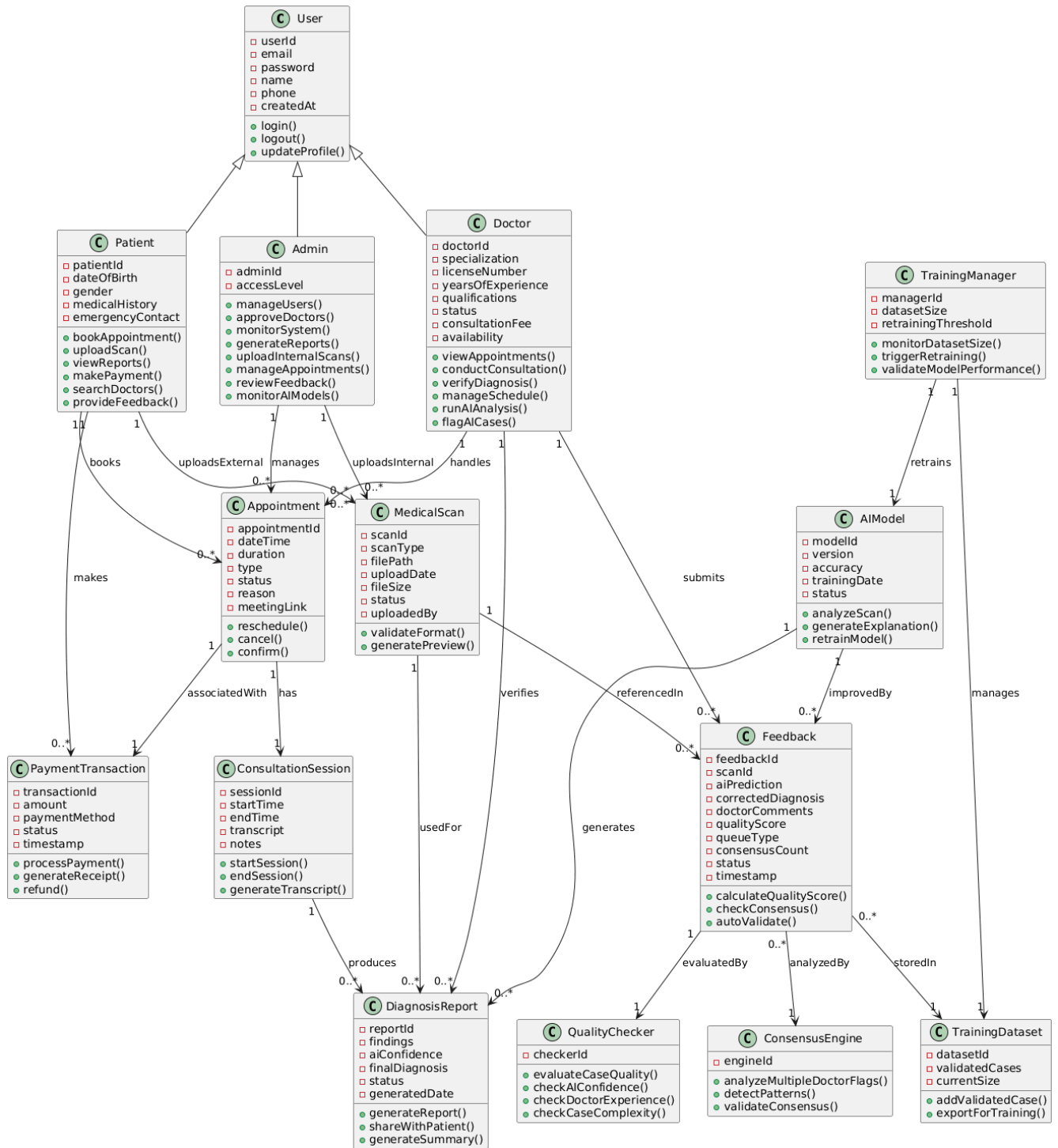


Figure 10: Class Diagram

Sequence Diagrams:

Sequence diagrams were developed for key interaction scenarios that involve complex object collaborations:

1. AI Diagnostic Execution:

Doctor selects patient scan and runs AI analysis, system processes scan through AI models, generates explainability visuals, and displays results to doctor.

Sequence diagram for AI Diagnostic Execution is given below:

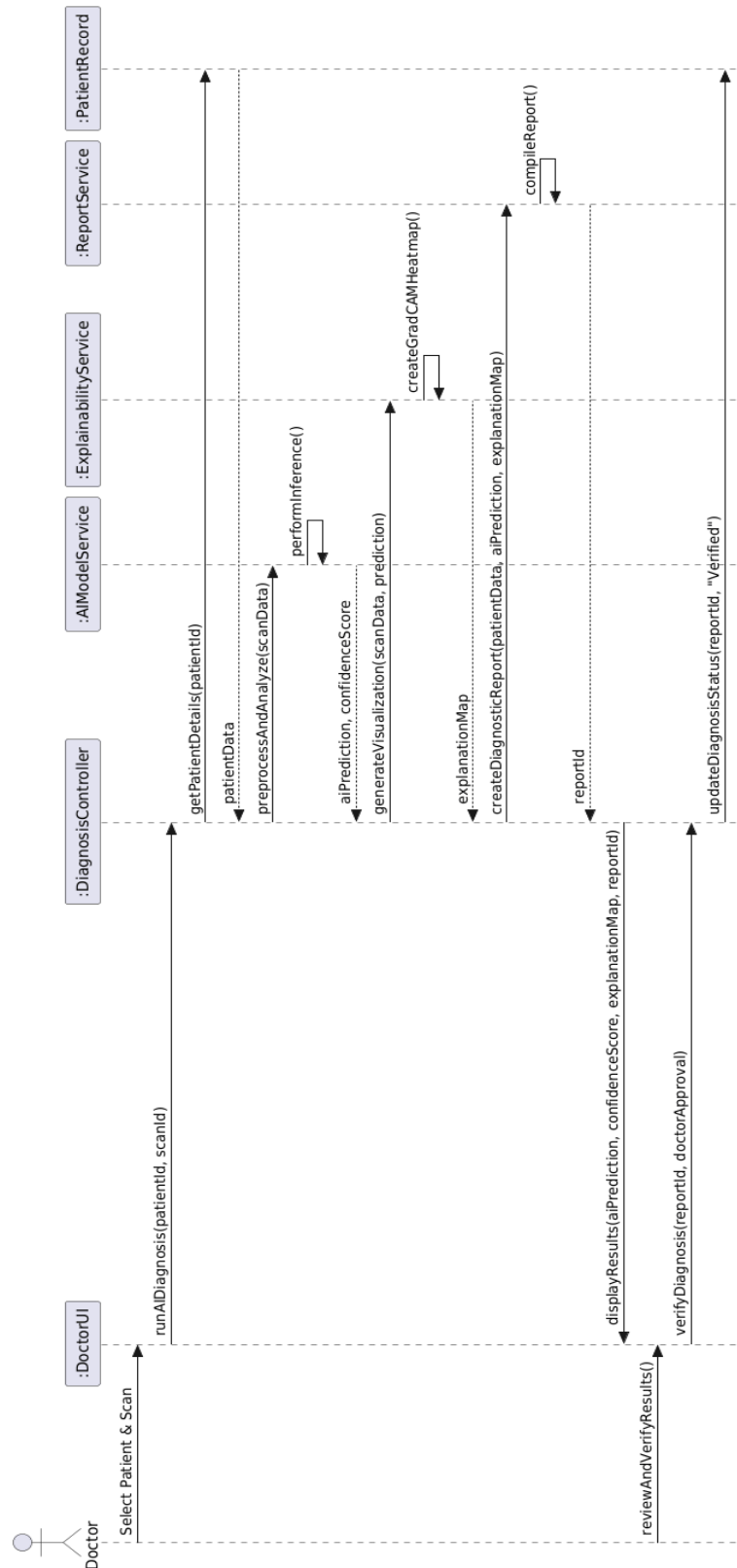


Figure 11: Sequence Diagram for AI Diagnostic Execution

2. Appointment Booking with Payment:

Patient searches doctors and selects slot, system processes payment through gateway, confirms booking and updates schedules with notifications.

Sequence diagram for Appointment Booking is given below:

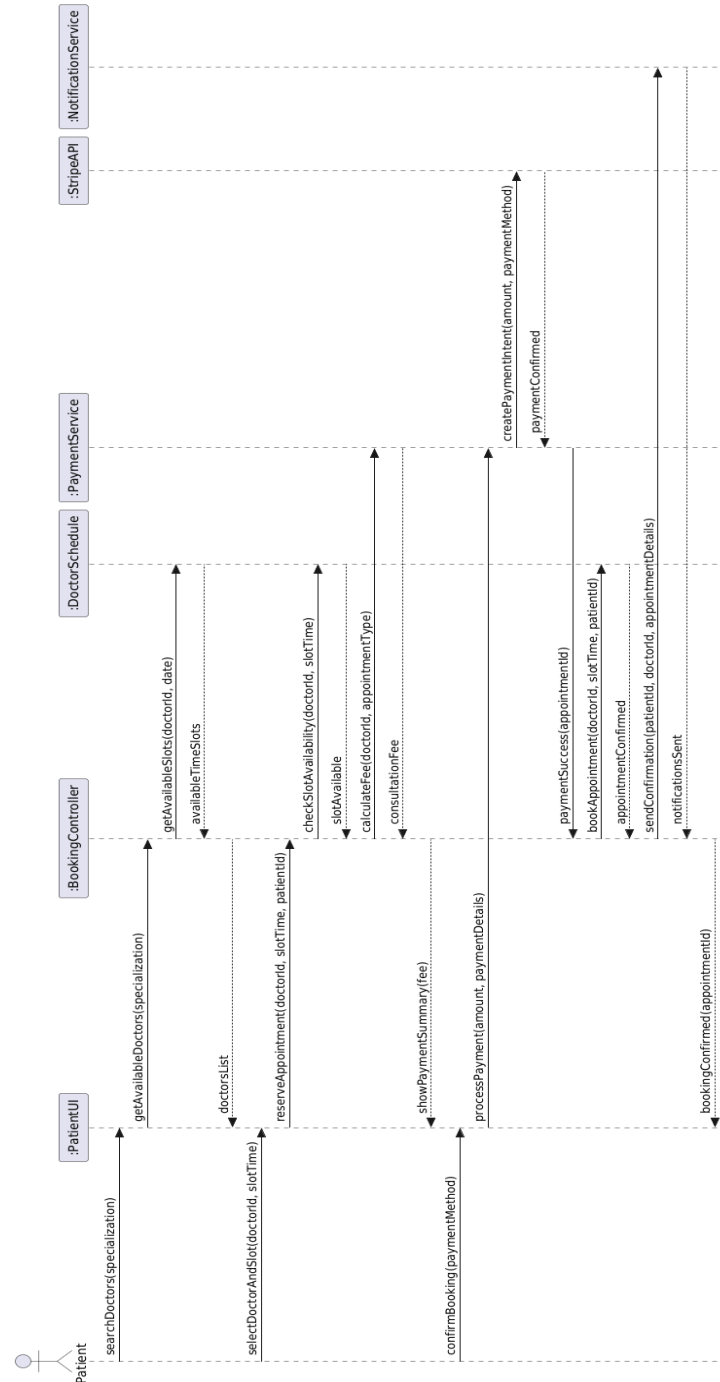


Figure 12: Sequence Diagram for Appointment Booking with Payment

3. Virtual Consultation Initiation:

Patient and doctor click join link for scheduled appointment, system creates secure video room and activates AI Note-Taker. Both users connect to video call while AI automatically transcribes their conversation in real-time.

Sequence diagram for Virtual Consultation Initiation is given below:

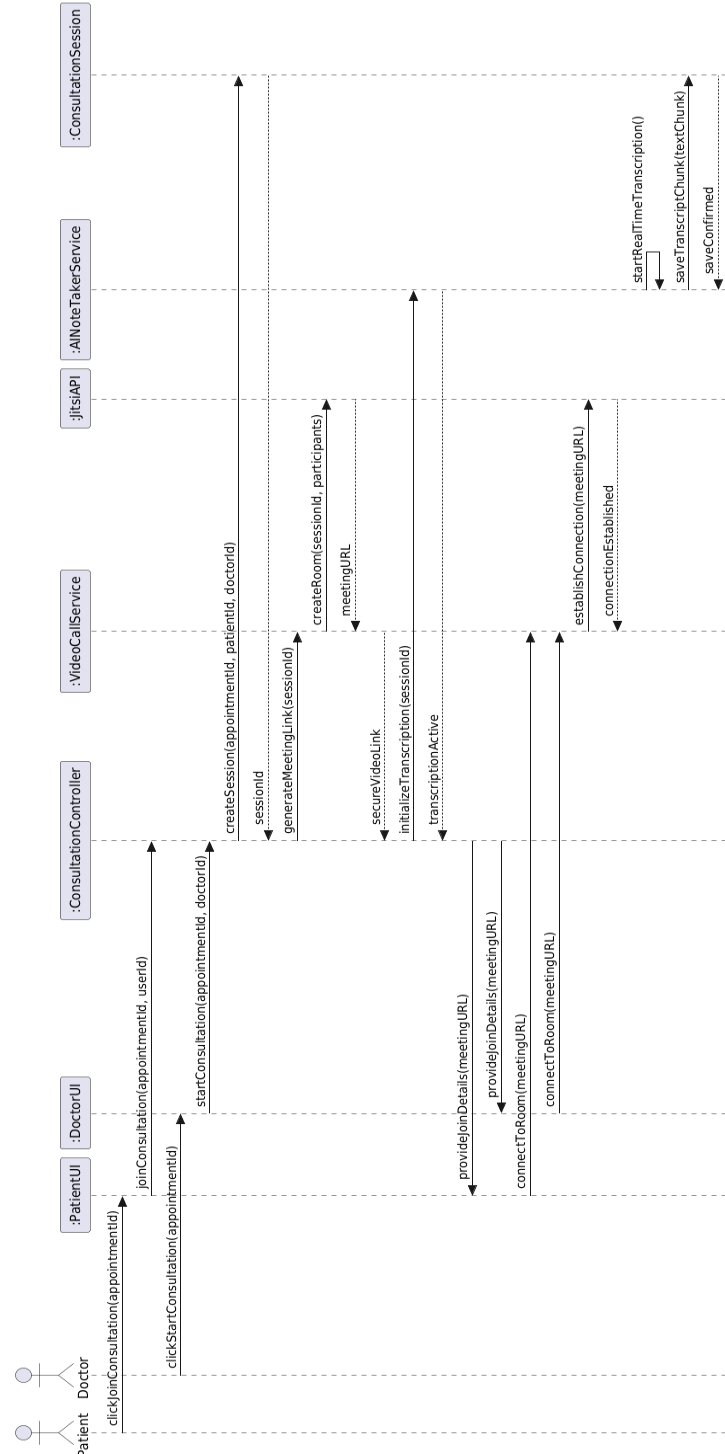


Figure 13: Sequence Diagram for Virtual Consultation Initiation

4. Patient Scan Upload & Validation:

Patient selects scan type and uploads file, system validates file format and size, stores scan securely in cloud storage, and links it to patient's medical record.

Sequence diagram for Patient Scan Upload & Validation is given below:

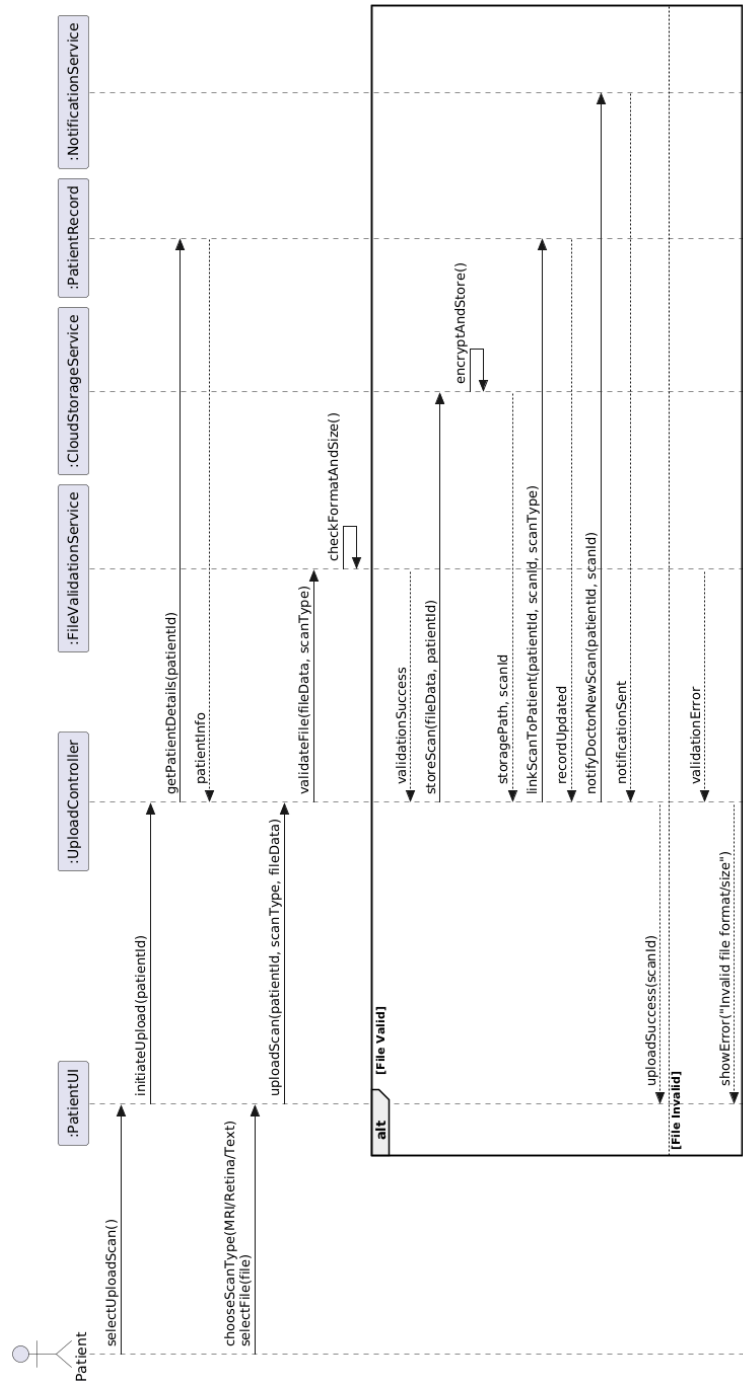


Figure 14: Sequence Diagram for Patient Scan Upload & Validation

5. Doctor Registration Approval:

Doctor submits registration with qualifications, admin reviews credentials and approves or rejects application. System updates doctor status and sends notification about approval decision.

Sequence diagram for Doctor Registration Approval is given below:



Figure 15: Sequence Diagram for Doctor Registration Approval

6. Self-Sufficient AI Error Flagging:

Doctor flags incorrect AI diagnosis, system automatically checks case quality and looks for similar corrections from other doctors. When multiple doctors flag the same issue, system validates it automatically and uses these cases to retrain and improve AI models.

Sequence diagram for Self-Sufficient AI Error Flagging is given below:

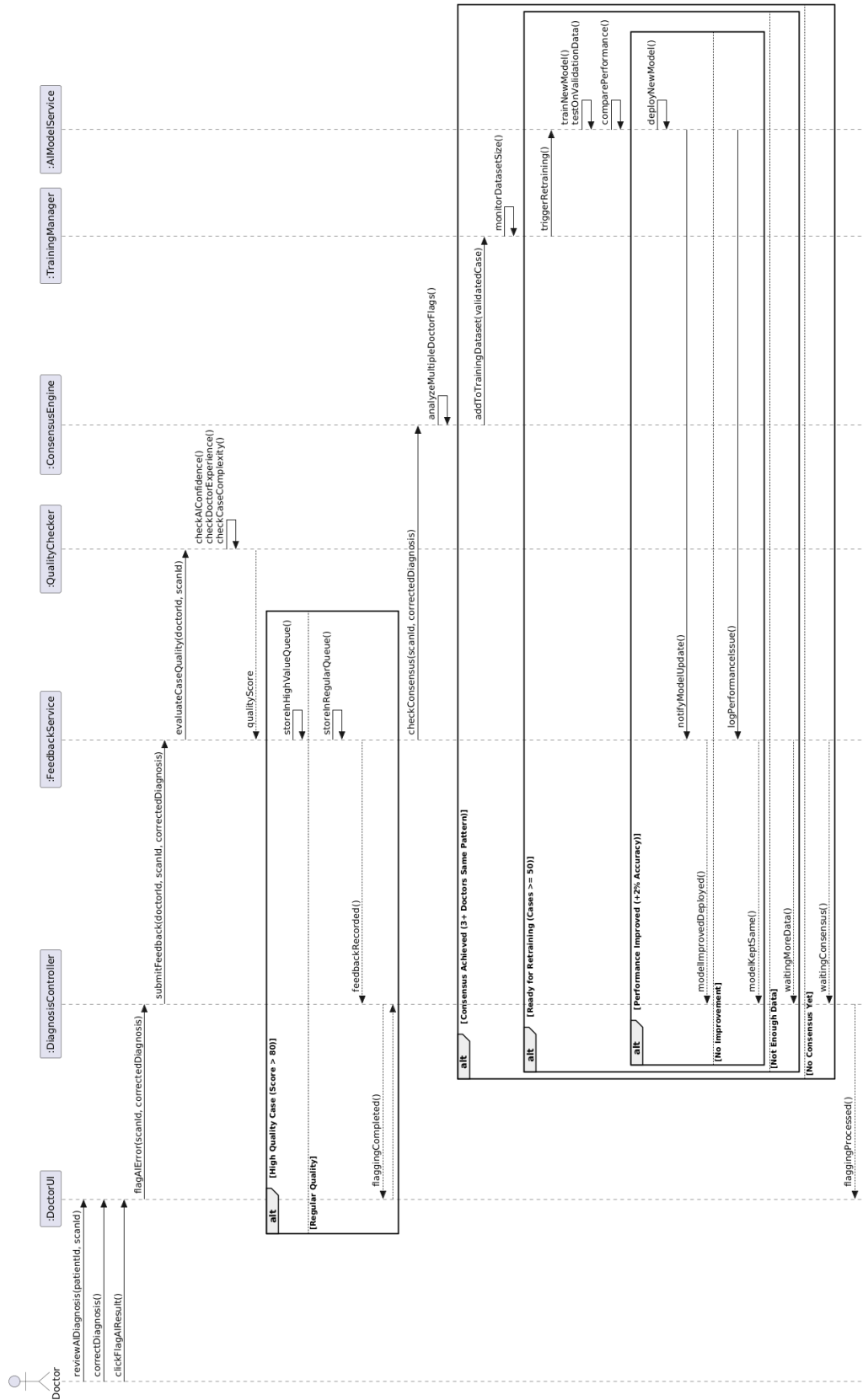


Figure 16: Sequence Diagram for Feedback Loop

State Transition Diagrams:

1. Appointment Object Lifecycle:

Appointment starts as available time slot, moves to pending payment when selected, becomes booked after successful payment, progresses to in-progress during consultation, and ends as completed or cancelled.

State diagram for Appointment Lifecycle is given below:

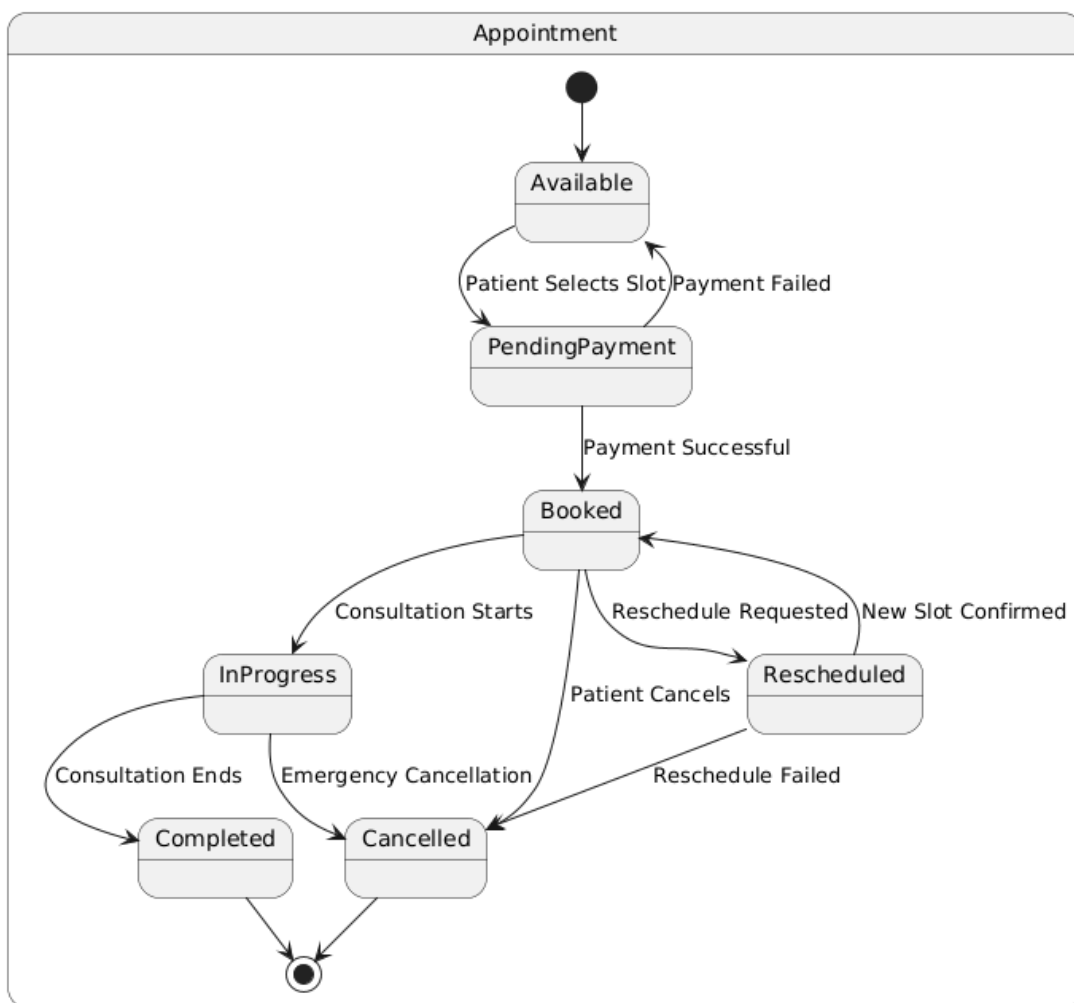


Figure 17: State Diagram for Appointment Object Lifecycle

2. Doctor Registration Object Lifecycle:

Doctor registration begins as draft, moves to pending approval after submission, becomes approved or rejected after admin review, and can be suspended or deactivated later.

State diagram for Doctor Registration is given below:

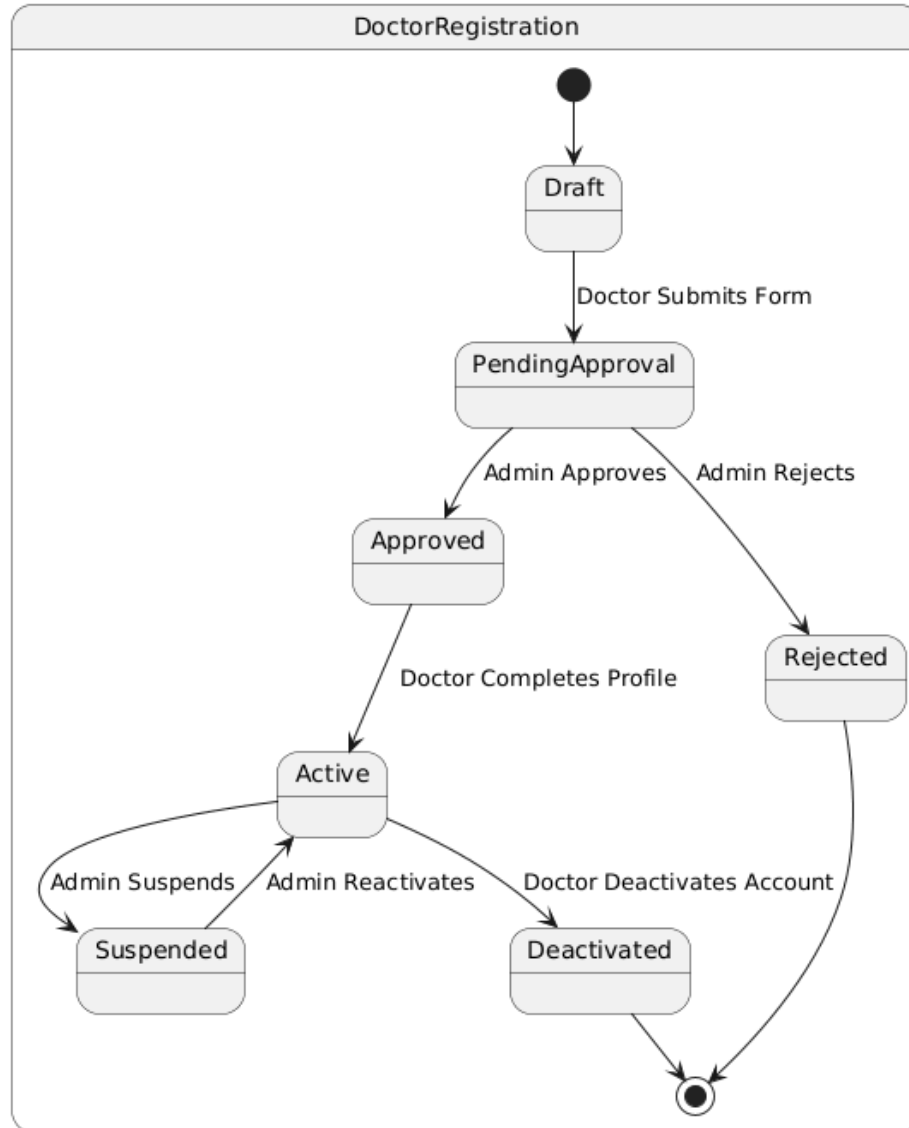


Figure 18: State Diagram for Doctor Registration

3. Diagnosis Report Object Lifecycle:

Report starts when scan is uploaded, moves to AI processing, generates results, goes under doctor review, and ends as verified, modified, or flagged based on doctor's decision.

State diagram for Diagnosis Report is given below:

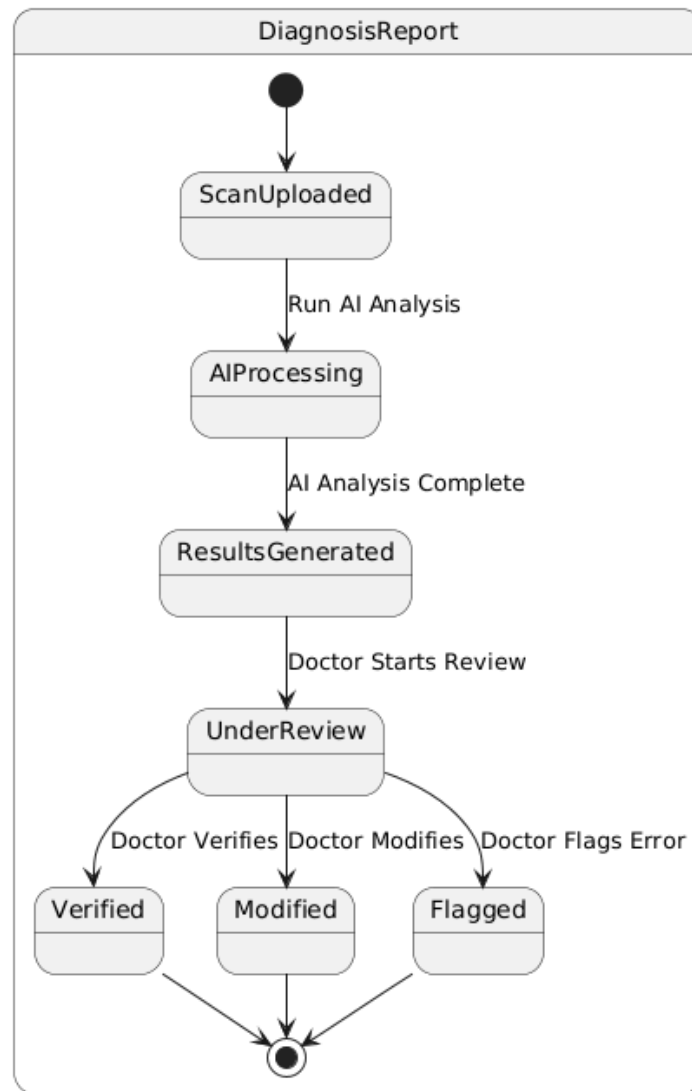


Figure 19: State Diagram for Diagnosis Report Object Lifecycle

4. Payment Transaction Object Lifecycle:

Payment starts when initiated, moves to processing during transaction, becomes successful or failed based on payment result, and can be refunded if needed.

State diagram for Payment Transaction is given below:

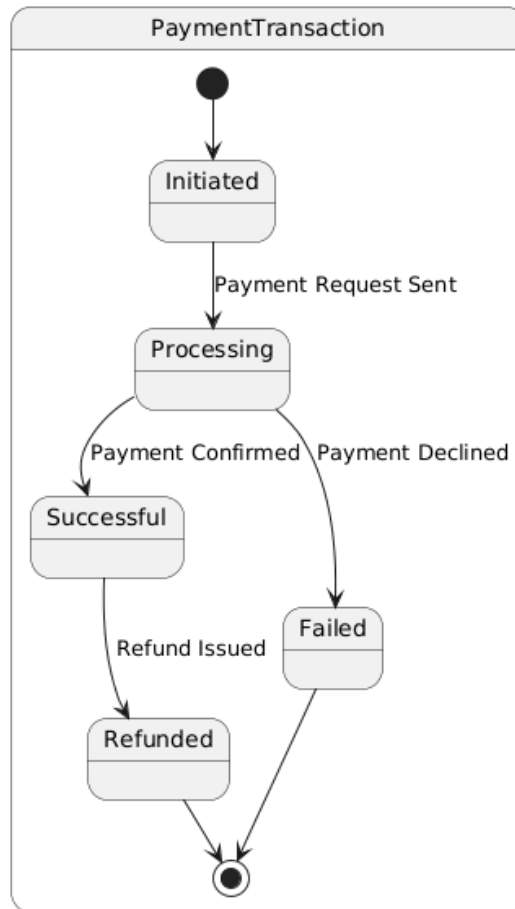


Figure 20: State Diagram for Payment Transaction Object Lifecycle

These state diagrams effectively capture the backend processes and event handling mechanisms critical to the system's operation.

3.5 Data Design

The data design of MediFusion Vision defines how healthcare information is structured, stored, and processed across the system's backend and database layers. The system adopts a hybrid data storage approach using PostgreSQL for structured relational data and cloud storage for medical imaging files, ensuring HIPAA-compliant data management with high security and scalability suitable for healthcare applications.

The backend, developed in Python (Flask) and Node.js, interacts with PostgreSQL, which defines schemas, enforces data validation, and supports complex relationships between medical entities. This relational design ensures data integrity while supporting complex healthcare workflows involving patients, doctors, diagnostics, and AI processing.

- **Major Data Entities and Their Relationships**

Entity	Description	Key Fields/Attributes	Relationships
User	Base class for all system users with common authentication and profile attributes.	userId, email, password, name, phone, createdAt, role	Parent of Patient, Doctor, Admin entities
Patient	Represents healthcare patients who book appointments, upload scans, and view diagnostic reports.	patientId, dateOfBirth, gender, medicalHistory, emergencyContact	Linked to Appointment, MedicalScan, DiagnosisReport, PaymentTransaction
Doctor	Represents medical professionals who conduct consultations and verify AI diagnoses.	doctorId, specialization, licenseNumber, yearsOfExperience, status, consultationFee	Linked to Appointment, DiagnosisReport, Feedback
Admin	Represents system administrators managing users, approvals, and system monitoring.	adminId, accessLevel	Manages all entities through administrative actions
Appointment	Represents scheduled consultations between patients and doctors, both virtual and physical.	appointmentId, dateTime, type, status, reason, meetingLink	Linked to Patient, Doctor, ConsultationSession, PaymentTransaction
MedicalScan	Stores medical imaging data including MRI and retinal scans with secure cloud storage references.	scanId, scanType, filePath, uploadDate, uploadedBy, status	Linked to Patient, Admin, DiagnosisReport, Feedback
DiagnosisReport	Contains AI-generated and doctor-verified diagnostic findings with confidence scores.	reportId, findings, aiConfidence, finalDiagnosis, status, generatedDate	Linked to MedicalScan, Doctor, ConsultationSession
ConsultationSession	Manages virtual consultation data including transcripts and AI-generated notes.	sessionId, startTime, endTime, transcript, notes	Linked to Appointment, DiagnosisReport
PaymentTransaction	Handles financial transactions for consultations with secure payment processing.	transactionId, amount, paymentMethod, status, timestamp	Linked to Patient, Appointment

Feedback	Stores doctor corrections for AI diagnoses enabling self-learning system improvement.	feedbackId, aiPrediction, correctedDiagnosis, qualityScore, consensusCount, status	Linked to Doctor, MedicalScan, QualityChecker, TrainingDataset
----------	---	--	--

- **Data Storage and Processing:**

Storage:

- **Structured Data:** All user records, appointments, medical data, transactions, and system logs stored in PostgreSQL with ACID compliance
- **Medical Imaging:** MRI scans, retinal images stored as encrypted files with metadata in database
- **AI Models:** Trained deep learning models stored as binary files with version tracking

Processing:

- Data interactions occur through database models that encapsulate schema definitions and CRUD operations
- AI diagnostic processing pipelines handle medical image preprocessing and model inference
- Automated feedback processing includes quality scoring and consensus detection
- Real-time consultation processing manages video streaming and AI note-taking

Security and Privacy:

- All patient data encrypted using AES-256 both at rest and in transit
- Role-Based Access Control (RBAC) ensures data access based on medical roles
- HIPAA and PDPA 2023 compliance through audit logging and access monitoring
- Payment information securely processed through integrated payment gateway
- Medical images access limited to authorized medical professionals only

2 Chapter 4: Implementation

This chapter presents the implementation details of the MediFusion Vision system, focusing on how the proposed architecture, deep learning models, and medical data pipelines were transformed into a fully functional AI-enabled diagnostic and patient-management platform. The implementation integrates the machine learning engine, backend services, database structures, and user-facing interfaces. Each component was developed using appropriate technologies, design patterns, and medical data standards to ensure diagnostic accuracy, performance, security, and clinical reliability.

The frontend (web and mobile) provides dedicated interfaces for patients, doctors, and

administrators. These interfaces interact with the backend through secure RESTful APIs that follow standardized response structures and medical-grade data protocols. The backend is developed using Node.js, Express.js, and Python-based AI microservices, organized under a modular layered architecture (Controllers, Routes, Models, Services, and Middlewares).

The AI engine runs the trained ResNet-18 and DenseNet-121 models deployed in a separate inference service built in Python (Flask/FastAPI). This service handles MRI preprocessing, prediction, confidence scoring, and explainability (Grad-CAM). The database layer uses MongoDB Atlas, which securely stores medical records, MRI metadata, AI results, patient history, doctor notes, and admin configurations.

Security mechanisms such as JWT authentication, bcrypt password hashing, HTTPS/TLS encryption, role-based access, and encrypted medical file uploads ensure the confidentiality and integrity of sensitive medical data. The system also integrates asynchronous operations for report generation, scheduled model updates, and alert notifications.

The following sections describe the implementation of core modules using structured pseudocode, explain the methodology used for training and evaluating AI models, and document the security, APIs, UI, and deployment strategy used in MediFusion Vision

4.1 Project Methodology & Algorithms

- This section outlines the step-by-step methodology adopted to implement MediFusion Vision, including the complete AI pipeline, backend workflow, and automation mechanisms. The methodology focuses on ensuring accurate medical diagnosis, efficient patient-doctor workflows, and secure handling of health information.

4.1.1 Project Methodology (Step-by-Step Approach)

- **Data Collection:**
 - **What:** Two separate MRI-based medical imaging datasets were used:
1. **Brain Tumor MRI Dataset** (Glioma, Meningioma, Pituitary, No-Tumor)
2. **Alzheimer's Disease MRI Dataset** (Mild, Moderate, Very Mild, Non-Demented)
 - **How:** Datasets were sourced from Kaggle, downloaded in JPEG/PNG format, and uploaded to the preprocessing pipeline. Metadata such as class labels and patient categories were mapped through structured directories.
 - **Why:** These datasets provide labeled MRI scans required to train the AI diagnostic engine. The images capture tumor shapes and Alzheimer-related tissue degeneration, making them optimal for deep learning-based classification.
- **Data Preprocessing:**

- **Resizing & Standardization:** All images were resized to a fixed dimension required by CNN models (224×224). Formats were unified to RGB to maintain consistency.
- **Normalization:** Pixel values were scaled to the range 0–1 for stable gradient flow and faster model convergence.
- **Class Balancing:** Oversampling, augmentation (rotation, flip, zoom), and balancing techniques ensured fairness across tumor and Alzheimer classes.
- **Why:** Medical imaging inconsistencies can reduce accuracy. Preprocessing enforces uniformity, removes irrelevant variation, and enhances the model’s ability to learn clinically relevant features.
- **Feature Extraction & Selection:**
 - **What:** High-level visual features such as:
 1. Tumor presence, texture, and structural boundaries
 2. Brain matter degeneration patterns linked to Alzheimer stages
 - **How:** No manual feature engineering was performed. Instead, deep CNN models, **ResNet-18** and **DenseNet-121**, automatically extracted:
 1. edges and contours
 2. lesion textures
 3. tissue abnormalities
 4. subtle degenerative patterns

Training and backpropagation ensure irrelevant features are ignored
 - **Why:** Hand-designed features are unreliable in medical imaging. CNNs offer superior extraction of complex patterns not visible through manual inspection.
- **AI Model Training & Core Algorithms:**
 - **Algorithm(s):** The MediFusion Vision diagnostic engine relies on two specialized deep learning architectures, each selected for its suitability to medical imaging:
 1. **ResNet-18** for **Brain Tumor Classification**
 2. **DenseNet-121** for **Alzheimer’s Disease Stage Classification**

These models form the core of the system’s multi-disease diagnostic capability and are optimized to detect subtle abnormalities in MRI scans.
 - **Training:** Both models were trained using a standard dataset split to ensure generalization:
 1. 70% Training set
 2. 20% Validation set
 3. 10% Test set
 - **Why:**
 1. **ResNet-18** was selected because its residual skip connections stabilize gradient flow, allowing the network to preserve edge-level and region-level features even in deeper layers. This makes it suitable for brain tumor detection, where strong contrast differences and irregular lesion boundaries must be captured reliably..
 2. **DenseNet-121** was chosen due to its dense connectivity pattern, which promotes feature reuse and captures subtle structural variations. This is particularly useful for Alzheimer’s disease detection, where patterns like cortical thinning and hippocampal volume loss require fine-grained feature extraction.
- **Advanced Techniques (if applicable)**

- **NLP:** The current phase of the system does not utilize NLP techniques such as tokenization, clinical text mining, or sentiment analysis. NLP support is planned for future modules like automated report drafting and patient-doctor interaction analysis.
- **Blockchain:**Blockchain was not included in this version of the system. Patient data integrity and privacy are ensured through standard encryption methods (AES-256 for stored data and JWT-based secure authentication).Blockchain-based immutable logs may be added in future upgrades.
- **Computer Vision:** MediFusion Vision analyzes brain MRI and retinal images using deep convolutional neural networks—ResNet-18 for tumor classification and DenseNet-121 for Alzheimer’s stage classification. Images are preprocessed with resizing, normalization, and data augmentation techniques (rotation, flipping, zoom) to ensure consistency and improve model generalization. These steps enable the models to automatically extract relevant features, such as intensity patterns, structural textures, and region-level variations, which support accurate classification and disease assessment..
- **Deployment**
 - **What:**The full system is deployed as a cloud-hosted Web + AI platform.
 - **How:**
 1. Frontend & Backend: **AWS EC2**
 2. AI Model Service: **AWS EC2 / Docker Container**
 3. Database: MongoDB Atlas
 - **Why:**Cloud deployment ensures high availability, scalability, and secure access for hospitals and remote patients.

4.1.2 Algorithm

The following algorithms form the core of MediFusion Vision’s diagnostic and workflow automation system. Each algorithm is expressed in structured pseudocode for technology-independent representation.

Table 4.1: Major Algorithms Used in MediFusion Vision

Algorithm Name	Details
Generic CNN-Based Classification Algorithm for Medical Imaging (Tumor & Alzheimer Detection)	Input: Dataset , ModelType (ResNet-18 / DenseNet-121), Epochs, BatchSize, LearningRate Output: TrainedModel, EvaluationMetrics Pseudocode: Input: Dataset (Images, Labels), ModelType (ResNet-18 / DenseNet-121), Epochs, BatchSize, LearningRate Output: TrainedModel, EvaluationMetrics (Accuracy, Precision, Recall, F1-score) Pseudocode: 1: procedure CNN_Classification(Dataset, ModelType, Epochs, BatchSize, LearningRate)

	<pre> 2: # Step 1: Data Preprocessing 3: For each image in Dataset do 4: Resize image to 224x224 5: Apply normalization (scale pixels 0-1 or standardize) 6: Apply data augmentation if training (rotation, flip, zoom) 7: end for 8: # Step 2: Split Dataset 9: Split Dataset into TrainSet, ValSet, TestSet 10: Ensure class balance (oversampling or weighted sampling if needed) 11: # Step 3: Initialize Model 12: if ModelType == "ResNet-18" then 13: Model ← Pretrained ResNet-18 14: Replace final layer to match number of classes 15: else if ModelType == "DenseNet-121" then 16: Model ← Pretrained DenseNet-121 17: Replace classifier to match number of classes 18: end if 19: # Step 4: Define Loss & Optimizer 20: LossFunction ← CrossEntropyLoss 21: Optimizer ← Adam(Model parameters, LearningRate) 22: # Step 5: Training Loop with Early Stopping 23: BestValAccuracy ← 0 24: PatienceCounter ← 0 25: for epoch in 1 to Epochs do 26: Model.train() 27: for each batch (Images, Labels) in TrainSet do 28: Outputs ← Model(Images) 29: Loss ← LossFunction(Outputs, Labels) 30: Backpropagate Loss 31: Update Model parameters using Optimizer 32: end for 33: # Step 6: Validation 34: Model.eval() 35: ValAccuracy ← Evaluate(Model on ValSet) 36: 37: if ValAccuracy > BestValAccuracy then 38: BestValAccuracy ← ValAccuracy 39: Save Model as BestModel </pre>
--	---

	<pre> 40: PatienceCounter ← 0 41: else 42: PatienceCounter ← PatienceCounter + 1 43: if PatienceCounter ≥ EarlyStoppingPatience then 44: break # Stop training early 45: end if 46: end if 47: end for 48: # Step 7: Testing 49: Load BestModel 50: Predictions ← Model(TestSet) 51: Compute EvaluationMetrics (Accuracy, Precision, Recall, F1- score) 52: return BestModel, EvaluationMetrics 53: end procedure </pre>
--	---

4.2 Training Results & Model Evaluation

- **Dataset Used:**

Brain Tumor Classification

- **Dataset Name:** Tumor_Split
- **Source:** Kaggle brain tumor MRI dataset
- **Classes:**
 - *glioma*
 - *meningioma*
 - *no_tumor*
 - *pituitary*
- **Dataset Structure:** Pre-split into Train / Validation / Test folders
- **Pre-processing:**
 - Resize to 224×224
 - Random horizontal flip
 - Random rotation (10°)
 - Normalize using ImageNet mean and std

Alzheimer Classification

- **Dataset Name:** Alzheimers_Split
- **Source:** Kaggle Alzheimer MRI dataset
- **Classes:**
 - *MildDemented*
 - *ModerateDemented*
 - *NonDemented*
 - *VeryMildDemented*
- **Dataset Structure:** Train / Validation / Test
- **Pre-processing:**
 - RandomResizedCrop (scale 0.8–1.0)
 - Random horizontal flip
 - 10° rotation
 - Brightness/contrast jitter
 - Normalize using ImageNet mean and std
- **Training Setup:**

Environment

- **Platform:** Kaggle GPU environment
- **Hardware:** NVIDIA Tesla T4 GPU
- **Framework:** PyTorch
- **Batch Size:** 32
- **Train/Val/Test:** Provided by dataset splits

Tumor Model (ResNet-18)

- **Base Model:** ResNet-18 (ImageNet pretrained)
- **Classifier:** Modified last FC → 4 classes
- **Optimizer:** Adam
- **LR:** 0.0001
- **Epochs:** 20 (stopped early via Patience=3)
- **Loss Function:** CrossEntropyLoss
- **Regularization:** Early stopping
- **Model Saving:** Best model saved on max validation accuracy

Alzheimer Model (DenseNet-121)

- **Base Model:** DenseNet-121 (trained from scratch)
- **Classifier:** Dropout(0.5) + Linear → 4-class output
- **Optimizer:** Adam

- **LR:** 1e-4
 - **Epochs:** 20 (ReduceLROnPlateau scheduler used)
 - **Class Imbalance Handling:** WeightedRandomSampler
 - **Regularization:** Learning-rate scheduling + dropout
 - **Early Stopping:** Patience = 5
- **Performance Metrics:**

Brain Tumor Model – Test Performance

- **Accuracy:** 98%
- **Precision / Recall / F1-score:**

Class	Precision	Recall	F1-score	Support
glioma	1.00	0.93	0.96	28
meningioma	0.93	1.00	0.97	28
no_tumor	1.00	1.00	1.00	20
pituitary	1.00	1.00	1.00	28

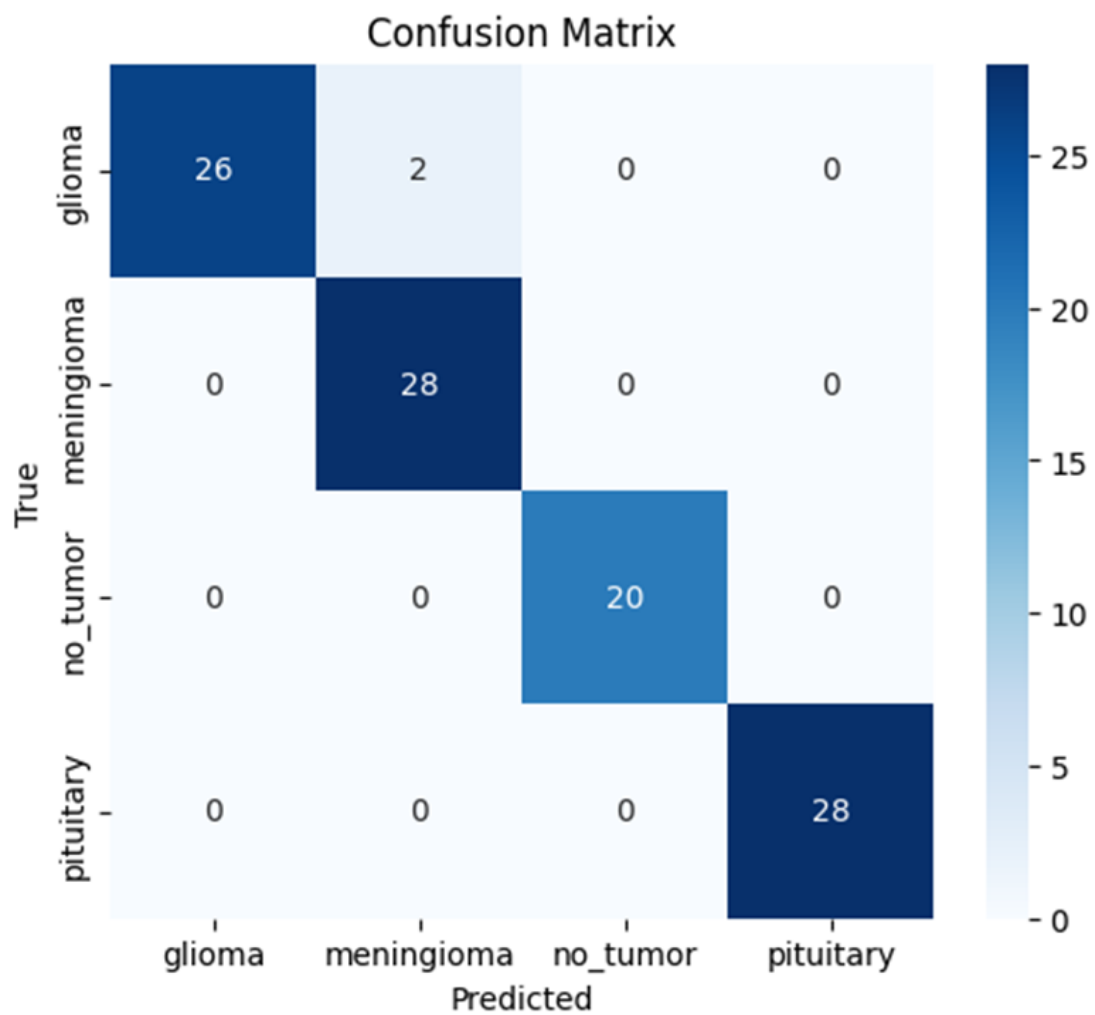
- **Macro Avg:** 0.98 precision, 0.98 recall, 0.98 F1
- **Weighted Avg:** 0.98 precision, 0.98 recall, 0.98 F1
- **Observations:** Very high performance, clean separation between classes, minimal misclassification.

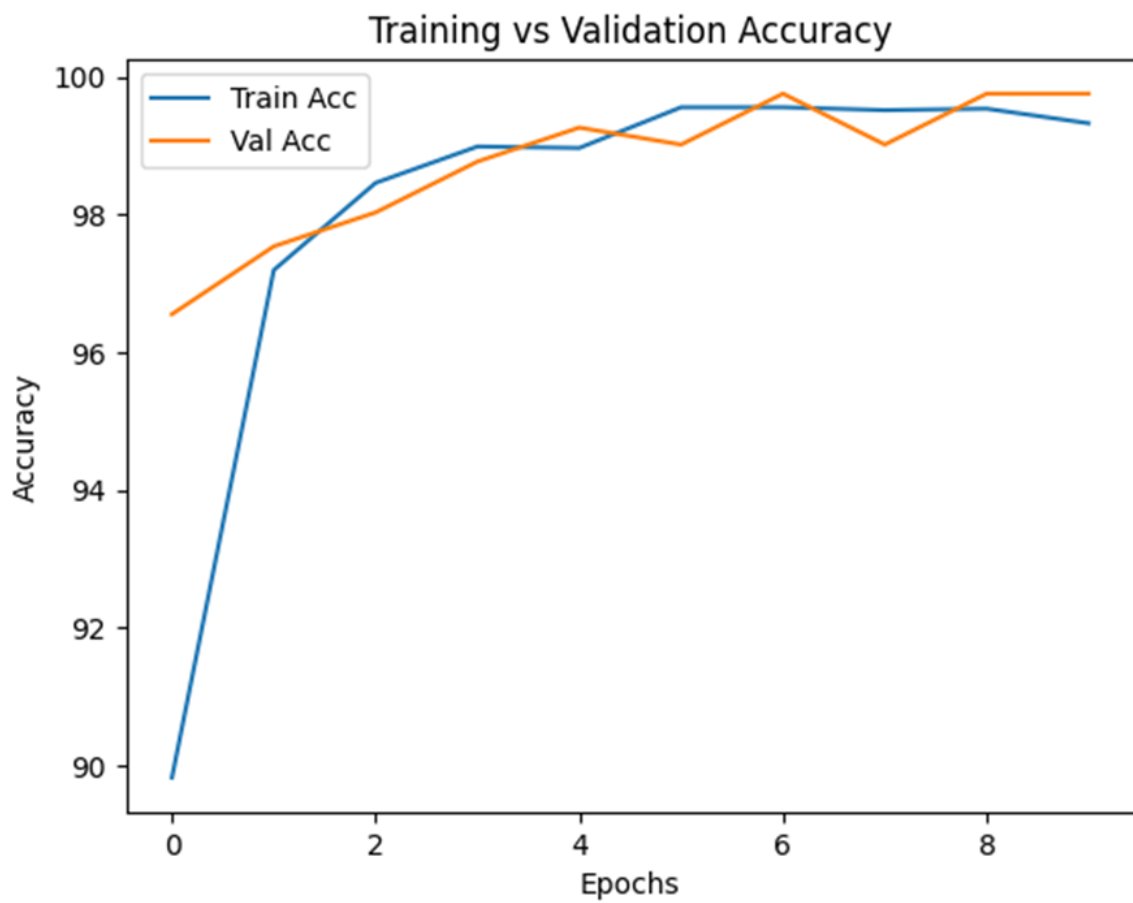
Alzheimer Model – Test Performance

- **Accuracy:** 99%
- **Precision / Recall / F1-score:**

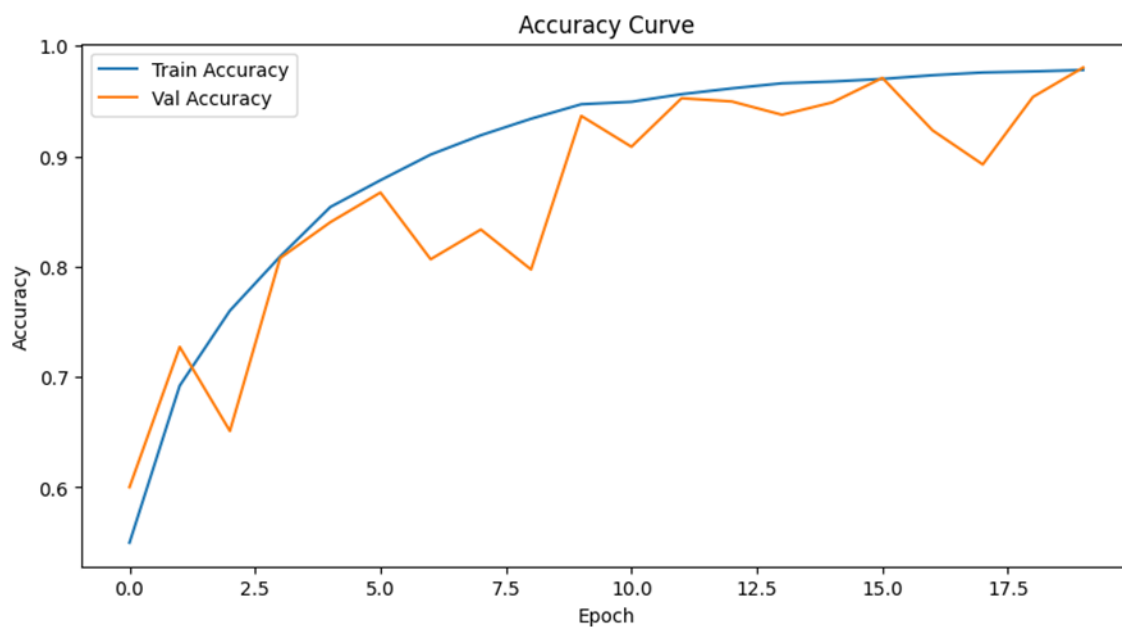
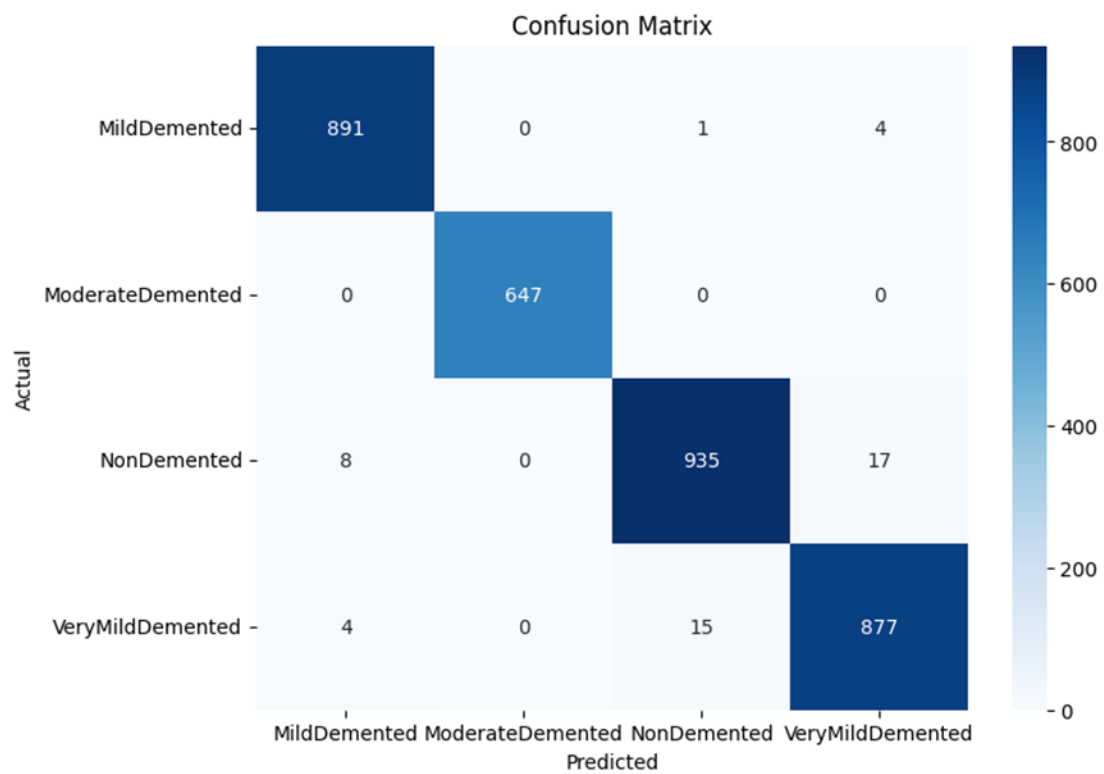
Class	Precision	Recall	F1-score	Support
MildDemented	0.99	0.99	0.99	896
ModerateDemented	1.00	1.00	1.00	647
NonDemented	0.98	0.97	0.98	960
VeryMildDemented	0.98	0.98	0.98	896

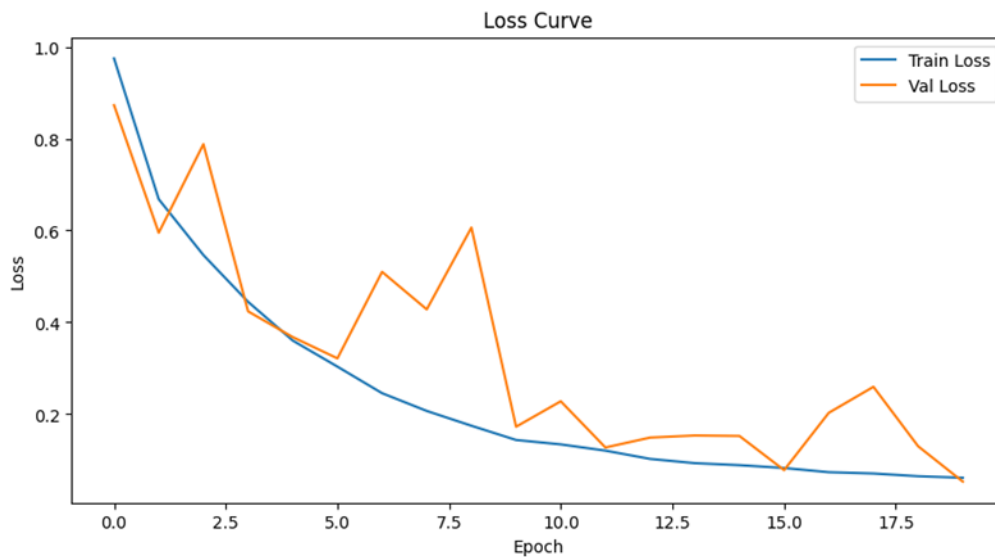
- **Macro Avg:** 0.99 precision, 0.99 recall, 0.99 F1
- **Weighted Avg:** 0.99 precision, 0.99 recall, 0.99 F1
- **Observations:** Model is highly stable due to robust augmentation + sampler.
- **Training Graphs and Confusion Matrix:**
Tumor Model:





Alzheimer Model





4.3 Security Techniques (if applicable)

- Techniques used for:
 - **Authentication:**
 1. Tokens include user ID, role, and time-limited expiry.
 2. Role-Based Access Control (RBAC) ensures controlled access:

Admin: full control (users, doctors, reports)

Doctor: diagnosis, patient files, consultations

Patient: appointments, reports, personal profile
 - **Encryption:**
 1. MRI file uploads are encrypted and stored using secure cloud storage
 2. No raw medical images or payment info are ever stored in plaintext
 - **Attack** **prevention:**

Table 4.2: Attack Prevention Techniques Applied in MediFusion Vision

Threat Type	Mitigation Technique
XSS (Cross-Site Scripting)	Input sanitization on API endpoints + strict validation in FastAPI
Injection Attacks (SQL/NoSQL)	Parameterized queries + ORM-level query filtering in the backend
Brute Force Attacks	Login rate-limiting + temporary account lockout after repeated failed attempts
Session Hijacking	Short-lived JWT tokens + secure token

	handling using Flutter Secure Storage
Data Exposure	Encrypted environment variables + strict role-based access control on sensitive APIs

4.4 External APIs/SDKs

Following are the APIs/SDKs used in the project implementation.

Table 4.3: APIs Used in MediFusion Vision:

Name of API and version	Description of API	Purpose of usage	Endpoints/Functions in used
Stripe (v20.0.0)	Payment processing platform	Secure online payments for appointment bookings	stripe.paymentIntents.create() stripe.refunds.create() stripe.webhooks.constructEvent()
Socket.io (v4.8.1)	Real-time bidirectional communication	Virtual consultation chat and notifications	io.on('connection') socket.join(room) socket.emit('receive_message')
Multer (v2.0.2)	File upload middleware	Medical scan file uploads	multer.diskStorage() upload.single('scan') File filtering
bcryptjs (v3.0.3)	Password hashing library	Secure password storage	wbcrypt.hash() bcrypt.compare()
jsonwebtoken (v9.0.2)	JWT token creation and verification	Stateless authentication	jwt.sign() jwt.verify()
Sequelize (v6.37.7)	PostgreSQL ORM	Database operations and migrations	Model definitions Associations Queries
React Router (v7.9.6)	Client-side routing	SPA navigation	<BrowserRouter> <Routes> useNavigate()
Framer Motion (v12.23.24)	Animation library	Smooth UI transitions and animations	motion.div animate props whileHover
Chart.js (v4.5.1)	Data visualization	Dashboard analytics and statistics	Line charts Bar charts Doughnut charts
React Dropzone (v14.3.8)	File upload interf		

4.5 User Interface

The MediFusion Vision UI is developed for three main user groups: Patients, Doctors, and Admins. The interfaces provide seamless workflows such as MRI upload, AI prediction display, patient history review, appointment management, and real-time reporting. Each interface is designed to be clean, minimally distracting, and medically appropriate.

Following are few examples of User Interfaces:

1.1.1 Login Screen

Login screen of our mobile app where user have to choose its role and its company.

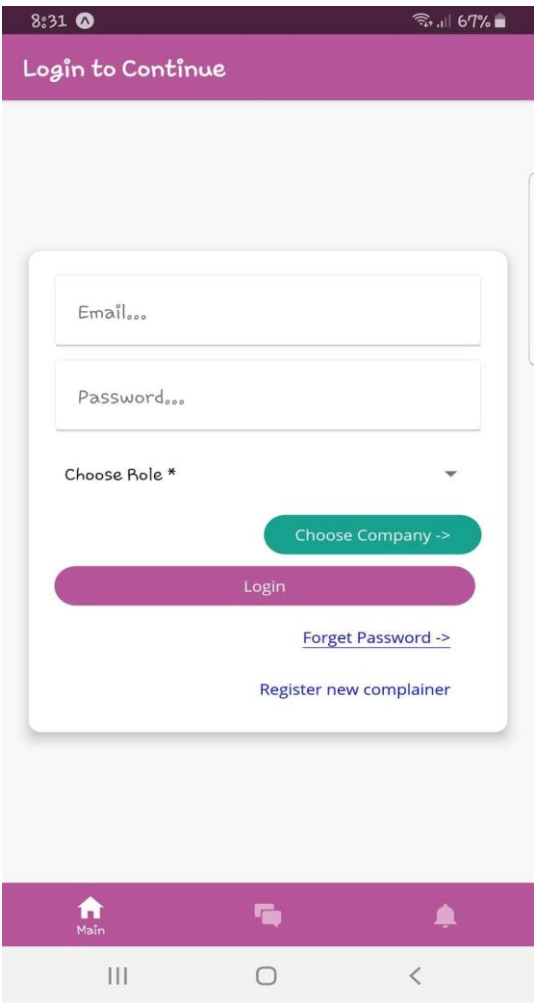
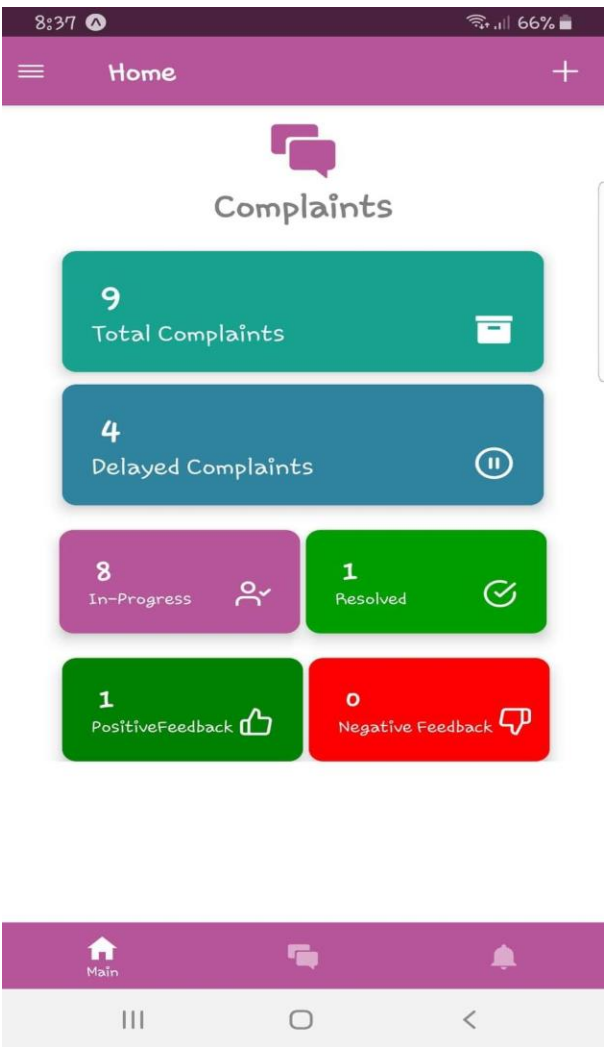


Figure 4.1 Login Screen

4.5.2 Home Screen

Home screen where total, delayed and Other complaints are shown.



5.3.3 Assignee Dashboard

Complain Assignee can view the graphs of month-wise complains, Resolved complains, summary and a list of submitted complains.

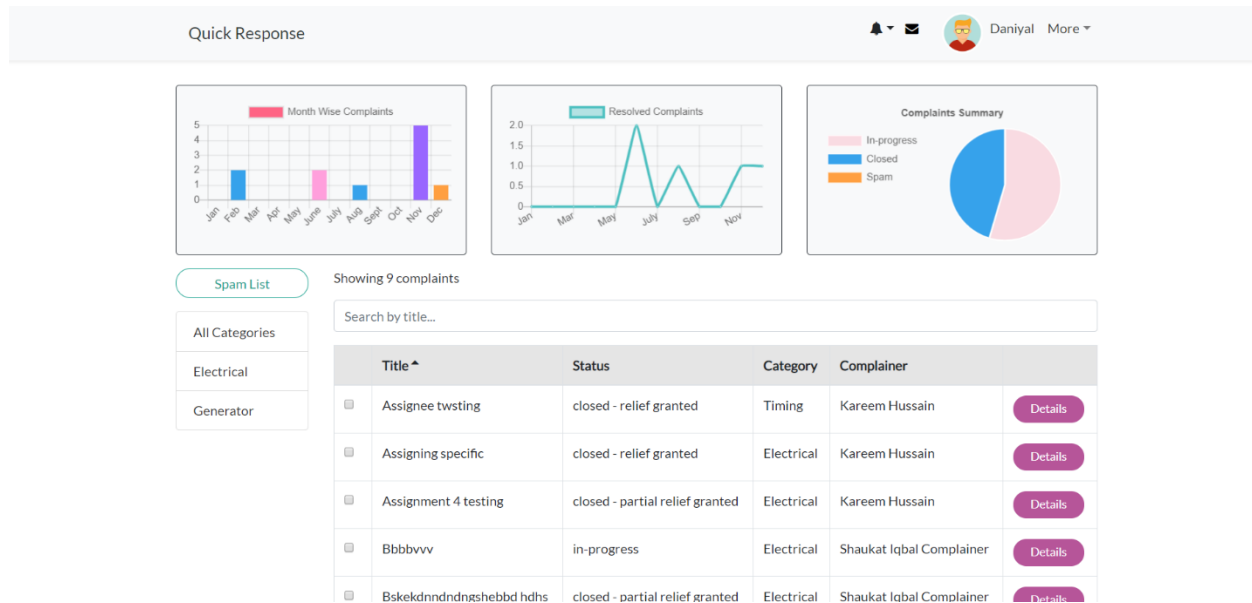


Figure 4.3 Assignee Dashboard

5.3.4 New Complaint

Complain Assignee and Complainer can create a new complain by providing Description, Category, Title, Location etc.

The 'New Complaint' form is a modal window with the following fields and options:

- Complaint Description:** A large text area for the complaint details.
- Selected Category:** A dropdown menu with 'General' selected.
- Title:** A text input field.
- Enter Location:** A text input field.
- Buttons:** 'Close' and 'Register' buttons at the bottom.

4.6 Deployment

The system is deployed using the following setup:

- **Backend Server:** Node.js v18+ with Express v5.1.0
- **Database Service:** PostgreSQL 14+ with Sequelize ORM v6.37.7
- **Authentication Service:** JWT (JSON Web Tokens) via jsonwebtoken v9.0.2
- **Frontend Framework:** React v19.2.0 with Vite v7.2.4 build system
- **Real-Time Communication:** Socket.io v4.8.1 for WebSocket connections
- **File Storage:** Local filesystem (server/uploads/) with Multer v2.0.2
- **Deployment Method:** Local development environment (cloud hosting not currently configured)

2 Chapter 5: Testing and Evaluation

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. The testing must be completed before it is deployed for use.

There are few types of testing which includes the unit testing, functional testing and integration testing.

You are required to perform each of these in-depth to ensure system quality.

5.1 Unit Testing

Unit testing verifies the **smallest testable components** of the software (e.g., individual functions, methods, or classes) in isolation. The purpose is to ensure that each unit performs as expected, independent of the full system.

At the FYP level:

- **Software Engineering students** may demonstrate automated unit tests using JUnit, PyTest, or similar frameworks.
- **Other programs (CS, AI, Cybersecurity, Data Science)** may show simplified unit-level tests where functions or algorithms are executed with inputs and outputs compared against expected results.

Unit Testing 1: validateEmail() function with valid and invalid credentials

Testing Objective: To ensure the email validation function works correctly with valid and invalid inputs.

N o.	Test case/Test script	Attribute and Value	Expected Result	Actual Result	Result
1	Call validateEmail() with valid email	"abc@gmail.com"	Validates as correct email (True)	True	Pass
2	Call validateEmail() with invalid email (missing @)	"abc.gmail.com"	Rejects input and returns False	False	Pass
3	Call validateEmail() with empty string	""	Rejects input and returns False	False	Pass
4	Call validateEmail() with invalid email (missing domain)	"abc@"	Rejects input and returns False	False	Pass
5	Call validateEmail() with invalid email (missing TLD)	"abc@domain"	Rejects input and returns False	False	Pass

6	Call validateEmail() with valid email (subdomain)	"user@mail.example.com"	Validates as correct email (True)	True	Pass
7	Call validateEmail() with null value	null	Rejects input and returns False	False	Pass
8	Call validateEmail() with undefined value	undefined	Rejects input and returns False	False	Pass

Unit Testing 2: validateScan() function with different file types and sizes
Testing Objective: To ensure the scan validation function correctly validates file types, sizes, and scan-specific requirements.

No.	Test case/Test script	Attribute and Value	Expected Result	Actual Result	Result
1	Call validateScan()) with valid MRI DICOM file	File: scan.dcm, Type: MRI, Size: 5MB	Returns empty errors array	[]	Pass
2	Call validateScan()) with invalid file type	File: document.txt, Type: MRI	Returns error: "Invalid file type"	["Invalid file type: .txt"]	Pass
3	Call validateScan()) with file exceeding size limit	File: large.jpg, Type: Retinal, Size: 60MB	Returns error: "Image file too large"	["Image file too large. Max: 50MB."]	Pass
4	Call validateScan()) with valid Retinal scan	File: retina.png, Type: Retinal, Size: 2MB	Returns empty errors array	[]	Pass
5	Call validateScan()) with null file	File: null, Type: MRI	Returns error: "No file uploaded"	["No file uploaded."]	Pass

6	Call validateScan() with Report type PDF	File: report.pdf, Type: Report, Size: 3MB	Returns empty errors array	[]	Pass
7	Call validateScan() with Report type image	File: image.jpg, Type: Report	Returns error: "Medical reports must be document files"	["Medical reports must be document files (.pdf, .doc, .txt)"]	Pass

Unit Testing 3: Password hashing and matching functions

Testing Objective: To ensure password hashing and matching functions work correctly for user authentication.

No.	Test case/Test script	Attribute and Value	Expected Result	Actual Result	Result
1	Create user with password "password123"	Password: "password123"	Password is hashed in database	Password stored as hash	pass
2	Match correct password	Entered: "password123", Stored: hashed	Returns true	true	pass
3	Match incorrect password	Entered: "wrongpass", Stored: hashed	Returns false	false	pass
4	Match empty password	Entered: "", Stored: hashed	Returns false	false	pass

Unit Testing 4: MRI Image Classification Function

Testing Objective: Ensure that the model correctly classifies a single MRI image input according to the expected label.

No.	Test case	Attribute and value	Expected Result	Actual Result	Result
1	Call <code>model(img)</code> with glioma MRI	Image of glioma brain tumor	Returns class glioma	glioma	True
2	Call <code>model(img)</code> with meningioma MRI	Image of meningioma brain tumor	Returns class meningioma	meningioma	True
3	Call <code>model(img)</code> with NonDemented Alzheimer MRI	Image of NonDemented patient	Returns class NonDemented	NonDemented	True
4	Call <code>model(img)</code> with VeryMildDemented MRI	Image of VeryMildDemented patient	Returns class VeryMildDemented	VeryMildDemented	True

5.2 Functional Testing

Functional testing validates that the **system modules** work correctly as a whole, ensuring that the developed system meets its specifications and requirements. Unlike unit testing, which focuses on internal functions, functional testing evaluates **user-facing features** through the UI or APIs.

Functional Testing 1: Login with different roles (Management, Patient, Doctor)

Objective: To ensure that the correct page with the correct navigation bar is loaded.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Login as 'Admin'	Username: admin@medifusion.com, Password: admin123	Admin dashboard with admin navigation bar is displayed	Redirected to Admin main page (/admin/dashboard)	Pass
2	Login as 'Doctor'	Username: doctor@medifusion.com, Password: doctor123	Doctor dashboard with doctor navigation bar is displayed	Redirected to Doctor dashboard (/doctor/dashboard)	Pass
3	Login as 'Patient'	Username: patient@medifusion.	Patient dashboard with patient	Redirected to Patient dashboard	Pass

		com, Password: patient123	navigation bar is displayed	(/patient/dashboa rd)	
4	Login with invalid credentials	Username: wrong@email.com, Password: wrongpass	Error message displayed: "Invalid email or password"	Error message shown	Pass
5	Login with unverified email	Username: unverified@email.com, Password: pass123	Error message: "Please verify your email before logging in"	Error message displayed	Pass
6	Login with locked account (5 failed attempts)	Username: locked@email.com, Password: wrongpass (5 times)	Account locked message displayed	"Account locked. Try again in 15 minutes."	Pass

Functional Testing 2: User Registration

Objective: To ensure that user registration works correctly for different roles and validates input properly.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Register as Patient	Name: "John Doe", Email: "john@example.com", Password: "password123", Role: "patient"	Registration successful, verification email sent	User created, verification token generated	Pass
2	Register as Doctor	Name: "Dr. Smith", Email: "smith@example.com", Password: "password123"	Registration successful, doctor profile created	User and doctor profile created	Pass

		3", Role: "doctor"			
3	Register with existing email	Email: "existing@example.com"	Error: "User already exists"	Error message displayed	Pass
4	Register with invalid email format	Email: "invalid-email"	Error: "Invalid email format"	Validation error displayed	Pass
5	Register with weak password (< 8 chars)	Password: "pass"	Error: "Password must be at least 8 characters"	Validation error displayed	Pass
6	Register with mismatched passwords	Password: "password123", Confirm: "password456"	Error: "Passwords do not match"	Error message displayed	Pass

Functional Testing 3: Appointment Booking

Objective: To ensure that appointment booking functionality works correctly with all validations.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Book appointment with valid data	Doctor: D001, Date: Future date, Time: Available slot	Appointment created successfully	Appointment created with status "pending"	Pass
2	Book appointment for past date	Date: Yesterday, Time: 10:00 AM	Error: "Cannot book appointments for past dates"	Error message displayed	Pass

3	Book appointment with unverified doctor	Doctor: Unverified doctor ID	Error: "Doctor is not yet verified"	Error message displayed	Pass
4	Book appointment with conflicting time slot	Doctor: D001, Date: Same, Time: Already booked	Error: "This time slot is already booked"	Error message displayed	Pass
5	Book appointment without complete profile	Patient profile incomplete	Error: "Please complete your profile before booking"	Error message displayed	Pass
6	Book virtual appointment	Type: "virtual"	Meeting link generated	Meeting link created	Pass

Functional Testing 4: Scan Upload and Analysis

Objective: To ensure that medical scan upload and AI analysis functionality works correctly.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Upload valid MRI scan	File: brain_scan.dcm, Type: MRI, Size: 10MB	Scan uploaded successfully	Scan saved to database	Pass
2	Upload scan with invalid file type	File: document.txt, Type: MRI	Error: "Invalid file type"	Error message displayed	Pass
3	Upload scan exceeding size limit	File: large.jpg, Size: 60MB	Error: "Image file too large. Max: 50MB"	Error message displayed	Pass

4	Request AI analysis for uploaded scan	Scan ID: valid_scan_id	AI analysis results returned	Analysis results with findings displayed	Pass
5	View scan results	Scan ID: valid_scan_id	Scan image and analysis displayed	Results page loaded correctly	Pass

Functional Testing 5: Classification Module

Objective: Ensure the model classifies any input image correctly, independent of its source.

No.	Test Case	Attribute and Value	Expected Result	Actual Result	Result
1	Tumor prediction	Input: Test MRI set (glioma, meningioma, pituitary, no_tumor)	Correct predictions for all images	Correct predictions for all images	Pass
2	Alzheimer prediction	Input: Test MRI set (MildDemented, ModerateDemented, NonDemented, VeryMildDemented)	Correct predictions for all images	Correct predictions for all images	Pass

5.3 Business Rules Testing

Decision table based testing technique is used to test business rules. The business rules were defined in FRs and Use Cases

Decision based testing uses a systematic approach where input and outputs are provided in tabular form. It is a precise and compact way to model complicated logic. The table contains conditions and actions are used for test cases where conditions as inputs and actions as outputs.

Business Rules Testing 1: Appointment Cancellation Refund Policy

Objective: To test the refund policy based on cancellation timing.
Decision Table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Hours until appointment > 24	Yes	Yes	No	No
Hours until appointment > 12	-	No	Yes	No
Hours until appointment <= 12	No	No	No	Yes
Action				
Refund Percentage	100%	100%	50%	0%
Refund Amount	Full	Full	Half	None

Test Cases:

No.	Test Case	Condition	Expected Result	Actual Result	Result
1	Cancel appointment 48 hours before	Hours: 48	Full refund (100%)	Refund: 100%	Pass
2	Cancel appointment 30 hours before	Hours: 30	Full refund (100%)	Refund: 100%	Pass
3	Cancel appointment 18 hours before	Hours: 18	Partial refund (50%)	Refund: 50%	Pass
4	Cancel appointment 6 hours	Hours: 6	No refund (0%)	Refund: 0%	Pass

	before				
5	Cancel appointment 1 hour before	Hours: 1	No refund (0%)	Refund: 0%	Pass

Business Rules Testing 2: Account Lockout Policy

Objective: To test the account lockout mechanism after failed login attempts.

Decision

Table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6
Failed attempts = 1	Yes	No	No	No	No	No
Failed attempts = 2	No	Yes	No	No	No	No
Failed attempts = 3	No	No	Yes	No	No	No
Failed attempts = 4	No	No	No	Yes	No	No
Failed attempts = 5	No	No	No	No	Yes	No
Failed attempts >= 5	No	No	No	No	No	Yes
Action						
Account Status	Active	Active	Active	Active	Locked	Locked
Lock Duration	-	-	-	-	15 min	15 min
Error Message	Invalid credentials	Invalid credentials	Invalid credentials	Invalid credentials	Account locked	Account locked

Test**Cases:**

No.	Test Case	Condition	Expected Result	Actual Result	Result
1	1 failed login attempt	Attempts: 1	Account active, error message shown	Error: "Invalid email or password"	Pass
2	2 failed login attempts	Attempts: 2	Account active, error message shown	Error: "Invalid email or password"	Pass
3	3 failed login attempts	Attempts: 3	Account active, error message shown	Error: "Invalid email or password"	Pass
4	4 failed login attempts	Attempts: 4	Account active, error message shown	Error: "Invalid email or password"	Pass
5	5 failed login attempts	Attempts: 5	Account locked for 15 minutes	Account locked, error: "Account locked. Try again in 15 minutes."	Pass
6	6+ failed login attempts	Attempts: 6	Account locked for 15 minutes	Account locked	Pass

Business Rules Testing 3: Doctor Verification Status**Objective:** To test the business rules for doctor verification and appointment booking.**Decision Table:**

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Doctor verification status =	Yes	No	No	No

"pending"				
Doctor verification status = "approved"	No	Yes	No	No
Doctor verification status = "rejected"	No	No	Yes	No
Doctor verification status = null	No	No	No	Yes
Action				
Can accept appointments	No	Yes	No	No
Can update PMDC number	Yes	No	No	Yes
Display status	Pending verification	Verified	Rejected	Not verified

Test Cases:

No.	Test Case	Condition	Expected Result	Actual Result	Result
1	Book appointment with pending doctor	Status: "pending"	Error: "Doctor is not yet verified"	Error message displayed	Pass
2	Book appointment with approved doctor	Status: "approved"	Appointment created successfully	Appointment booked	Pass
3	Book appointment with rejected doctor	Status: "rejected"	Error: "Doctor is not verified"	Error message displayed	Pass

4	Update PMDC for pending doctor	Status: "pending"	PMDC number updated	Update successful	Pass
5	Update PMDC for approved doctor	Status: "approved"	PMDC number updated	Update blocked	Pass

Business Rules Testing 4: Appointment Time Validation

Objective: To test the business rules for appointment time slot validation.

Decision

Table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5
Date is in past	Yes	No	No	No	No
Date is today, time < current + 30min	No	Yes	No	No	No
Date is today, time >= current + 30min	No	No	Yes	No	No
Date is future, within working hours	No	No	No	Yes	No
Date is future, outside working hours	No	No	No	No	Yes
Action					
Allow booking	No	No	Yes	Yes	No
Error	"Cannot book	"Time slot	-	-	"Outside

message	for past dates"	too soon"			working hours"
---------	-----------------	-----------	--	--	----------------

Test

Cases:

No.	Test Case	Condition	Expected Result	Actual Result	Result
1	Book appointment for yesterday	Date: Past	Error: "Cannot book appointments for past dates"	Error displayed	Pass
2	Book appointment for today, 15 min from now	Time: Current + 15min	Error: "Time slot too soon"	Error displayed	Pass
3	Book appointment for today, 45 min from now	Time: Current + 45min	Appointment created	Booking successful	Pass
4	Book appointment for future date, within hours	Date: Future, Time: 10:00 AM (within hours)	Appointment created	Booking successful	Pass
5	Book appointment for future date, outside hours	Date: Future, Time: 11:00 PM (outside hours)	Error: "Outside working hours"	Error displayed	Pass

Detailed example is as given in Appendix E.

5.4 Integration Testing

Integration testing verifies that different modules of the system work together correctly. Unlike unit testing (which checks isolated functions) and functional testing (which checks features from

a user's perspective), integration testing focuses on the **interfaces, linkages, and data flow between modules developed by different team members.**

Integration Testing 1: Scheduling Patient Appointment

Testing Objective: To ensure the scheduling is being done correctly and the interface between module 'Patient/Doctor Management' and module 'Appointment/Scheduling' is running correctly.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Create Appointment (Patient ↔ Doctor ↔ Scheduler)	Doctor schedule, Patient's preferred date/time	Appointment record created with correct doctor, patient, and date/time	Appointment created successfully	Pass
2	Update Appointment (Scheduler ↔ Database ↔ Notification)	Select new date/time	Appointment updated and linked records (database + notification) reflect change	Appointment updated successfully	Pass
3	Cancel Appointment (Patient ↔ Payment ↔ Notification)	Cancel appointment with payment	Appointment cancelled, refund processed, notifications sent	Appointment cancelled, refund calculated, notifications logged	Pass
4	View Appointments (Patient ↔ Doctor ↔ Database)	Request appointment list	All appointments for user displayed with doctor/patient details	Appointments retrieved with correct associations	Pass

Integration Testing 2: Scan Upload and AI Analysis Integration

Testing Objective: To ensure the integration between Scan Upload module, AI Analysis module, and Database module works correctly.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Upload Scan (Patient ↔ File Storage ↔ Database)	Patient uploads MRI scan file	Scan file stored, database record created	Scan uploaded and record created	Pass
2	Analyze Scan (Database ↔ AI)	Request AI analysis for scan	AI analysis performed, results stored in database	Analysis results	Pass

	Service ↔ Results Storage)			generated and saved	
3	View Results (Patient ↔ Database ↔ AI Results)	Patient requests scan results	Scan image and AI analysis displayed	Results page shows scan and findings	Pass
4	Doctor Reviews Analysis (Doctor ↔ Database ↔ AI Results)	Doctor views patient scan analysis	AI analysis displayed with option to add notes	Analysis displayed, doctor can add notes	Pass

Integration Testing 3: User Registration and Profile Completion

Testing Objective: To ensure the integration between Authentication module, User Management module, and Profile Management module works correctly.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Register User (Auth ↔ User Management ↔ Email Service)	New user registration	User created, profile initialized, verification email sent	User created, email token generated email sent oaded and record created	Pass
2	Verify Email (Email Service ↔ Auth ↔ User Management)	User clicks verification link	Email verified, user status changed to active	Status updated to "active"	Pass
3	Complete Patient Profile (Auth ↔ Profile Management ↔ Database)	Patient fills profile form	Profile data saved, user can now book appointments	Profile saved successfully	Pass
4	Complete Doctor Profile (Auth ↔ Profile Management ↔ Verification Queue)	Doctor fills profile with PMDC	Profile saved, doctor added to verification queue	Profile saved, status: "pending"	Pass

Integration Testing 4: Payment Processing Integration

Testing Objective: To ensure the integration between Appointment module, Payment Gateway module, and Notification module works correctly.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Process Payment (Appointment ↔ Payment Gateway ↔ Database)	Patient pays for appointment	Payment processed, appointment confirmed, payment record created	Payment successful, appointment status: "confirmed"	Pass
2	Payment Failure (Payment Gateway ↔ Notification ↔ Database)	Payment fails	Error message displayed, appointment remains pending	Error displayed, appointment status unchanged	Pass
3	Refund Processing (Cancellation ↔ Payment Gateway ↔ Database)	Cancel appointment with refund	Refund processed, payment status updated	Refund calculated and processed	Pass
4	Payment Receipt (Payment ↔ Email Service ↔ Patient)	Payment successful	Receipt email sent to patient	Email notification sent (mock)	Pass

